

Hafta 6 – Çalışma Zamanı Uygulama Öz Koruması (RASP)

Tarih	23.10.2026
Öğrenme çıktıları	ÖÇ.3 (kod sağlamlaştırma (hardening) tekniklerini – RASP dâhil – açıklar ve C/C++, Java için uygular)
Süre	3 saat (3 × 50 dakika)
Ön bilgi	C'de işaretçi, dizi, dosya; Hafta 1 'den bellek düzeni ve güvenli silme; Hafta 3 'ten AES-GCM/HMAC/HKDF ve güvenlik kabuğu; Hafta 4 'ten yama ve tersine mühendislik kavramları
Uygulamalar	code/week-06 – 8 demo; Windows'ta <code>.\demo.ps1</code> , WSL/Linux'ta <code>sh demo.sh</code>



Bu haftanın sonunda şunları yapabileceksiniz

- RASP** (Runtime Application Self-Protection) kavramını tanımlamak; **algılama** → **savunma** → **caydırma** üçlüsünü ve RASP'in klasik dış korumalardan (WAF, ağ güvenlik duvarı) farkını açıklamak.
- Güvenilmez cihaz (MATE)** saldırgan modelini benimsemek: cihazın sahibi saldırgandır; root/hata ayıklayıcı/kanca bekleriz.
- Bir uygulamanın **kendi bütünlüğünü** çalışma zamanında doğrulamasını (self-hashing, HMAC) kurmak ve **yamayı (patch)** tespit etmek; "checker'in kendisi de yamalanabilir" sınırını tartışmak.
- Hata ayıklayıcı** (ptrace/TracerPid, IsDebuggerPresent, PEB), **ortam/emülatör** (CPUID, zamanlama) ve **kanca/enstrümantasyon** (LD_PRELOAD, dlsym/dladdr, Frida) algılamayı çalışan örnekte göstermek.
- Kök/ayrıcalıklı ortam** göstergelerini (root, `su` , yükseltilmiş yetki) okumak ve bunların **sinyal, kanıt değil** olduğunu; yanlış pozitif tartışmak.
- Bileşen/paket imza ve özet doğrulamasını** – Android'deki APK imza doğrulamasının genel hâlini – kurmak ve **yeniden paketlemeyi (repackaging)** yakalamak.
- Kontrol akışı sayacı** ile tek bir `if` kontrolünün atlanamayacağı bir yapı kurmak (skip saldırısına direnç).
- Tepki (response) politikasını** (fail-closed, güvenli silme, sahte/decoy çıktı, cihaz/sürüm bağlama) tasarlamak ve bütün parçaları bir **RASP motorunda** birleştirmek.

Ders akışı (öğretim üyesi için zaman planı)

Zaman	Bölüm	Ne yapılıyor
0:00–0:10	1	RASP nedir, algılama/savunma/caydırma, MATE modeli, laboratuvar kontrolü
0:10–0:25	2	RASP mimarisi: denetimler ne zaman, nerede, nasıl karar verir
0:25–0:50	3	Bütünlük denetimi (self-hashing); Demo 1 (HMAC yama tespiti)
0:50–1:00	Ara	
1:00–1:20	4	Hata ayıklayıcı algılama; Demo 2 (ptrace/PEB)
1:20–1:35	5	Ortam/emülatör algılama; Demo 3 (CPUID + zamanlama)
1:35–1:50	6	Kanca/enstrümantasyon; Demo 4 (LD_PRELOAD, dladdr) — sınıf etkinliği
1:50–2:00	Ara	
2:00–2:15	7	Dinamik bellek koruması ve bellek izleme tespiti
2:15–2:30	8	Kök/ortam göstergesi (Demo 7); bileşen imzası (Demo 6)
2:30–2:45	9	Kontrol akışı sayacı; Demo 5 (skip saldırısı)
2:45–2:57	10	Tepki politikası + cihaz bağlama; Demo 8 (RASP motoru)
2:57–3:00	11+	Sınırlar, proje adımı (S10), kapanış



Laboratuvarı önceden hazırlayın

Demolar hem **Windows** (Visual Studio 2022 derleyicisi, ek kurulum yok) hem **WSL/Linux** (GCC) üzerinde **aynı kaynaktan** derlenir. Bütünlük ve imza demoları kriptoloji için ortak `cen429_kripto.h` başlığını kullanır (Linux'ta **OpenSSL**, Windows'ta **BCrypt/CNG**); ek kurulum gerekmez. **Demo 4 yalnız Linux/WSL'de** çalışır (LD_PRELOAD).

```
cd code
.\build.ps1
cd week-06\01-butunluk-hmac
.\demo.ps1      # ya da demo.cmd dosyasına çift tıklayın

cd code
./build.sh
cd week-06\01-butunluk-hmac
sh demo.sh
```

Dosya > Aç > Klasör > code/ deyin; üstteki yapılandırmadan "**Windows (MSVC)**" ya da "**WSL (GCC)**" seçin; **Derle > Tümünü Derle**. Anti-debug demosunu **F5** ile (hata ayıklayıcıda) çalıştırırsanız algılamanın tetiklendiğini görürsünüz.

Kurulum ayrıntıları ve güvenlik kuralları: [code/README.md](#).



Etik kural — bu derste her hafta geçerli

Bu hafta uygulamanın **kendini** koruma tekniklerini ve bunların **atlatılmasını** görüyoruz. Bütün demolar yalnız kendi klasörlerinde dosya üretir, **yönetici yetkisi istemez**, sistem ayarına dokunmaz. "Yama" adımı ikili dosyanın **kendi kopyası** üzerinde yapılır; hata ayıklayıcı yalnız **demonun kendi sürecine** bağlanır; **LD_PRELOAD** yalnız **tek bir demo komutunu** etkiler. Öğrendiğiniz teknikleri **yalnız kendi bilgisayarınızda ve verilen demolar üzerinde** deneyin. Başkasının uygulamasını izinsiz tersine çevirmek ya da korumasını kırmak yasadışı olabilir.

0. Başlamadan önce

Bu bölüm haftaya hazırlık içindir. Önce bu haftanın dayandığı önceki konuları kısaca hatırlatır; sonra bu haftanın kavramlarını birer cümleyle tanımlayıp her birini ayrıntılı anlatıldığı bölüme bağlar.

Önceki haftalardan gelenler

- **Beyaz kutu saldırgan modeli** — cihazın sahibi olan kullanıcının aynı zamanda potansiyel saldırgan olduğu; belleği okuyup hata ayıklayıcı bağlayabildiği ve kodu değiştirebildiği model ([Hafta 1, §3](#); "beyaz kutu saldırganın altı yolu" tablosuna da bakınız: [Hafta 1, §5](#)). Bu hafta bu modeli **MATE** adıyla RASP'in tek saldırgan modeli yapıyoruz (§1.1).
- **Özet, MAC ve HMAC** — özet bir veriden hesaplanan sabit boyutlu, tek yönlü bir parmak izidir; MAC/HMAC ise paylaşılan bir gizli anahtarla mesajın değişmediğini kanıtlar ([Hafta 3, §2.4](#)). Bu hafta HMAC-SHA-256'yı uygulamanın kendi kodunu doğrulamak (§3) ve bir kontrol akışı zinciri kurmak (§9) için kullanıyoruz.
- **Ana sırdan anahtar türetme (HKDF)** — RFC 5869 HKDF, bir ana sırdan Extract ve Expand adımlarıyla yeni anahtarlar türeten fonksiyondur ([Hafta 3, §7](#)). Bu hafta aynı iki adımı, bir ana sır yerine bir **cihaz parmak izinden** anahtar türetmek için elle izliyoruz (§10.1).

- **Cihaz bağlama** — bir sırrın yalnızca belirli bir cihazda anlamlı olması, güvenlik kabuğunun en iç katmanıdır ([Hafta 3, §13.1](#)). Bu hafta bu katmanı bir RASP tepki politikasının parçası yapıyoruz (§10).

Bu haftanın kavram haritası

Kavram	Bir cümlede	Ayrıntısı
RASP (algıla/savun/caydır)	RASP, uygulamanın kendi içine gömülen, çalışma zamanında kendini izleyen ve saldırıya algılama, savunma ve caydırma ile tepki veren korumalardır.	§1
MATE saldırgan modeli	MATE ("Man-At-The-End"), saldırganın uygulamanın çalıştığı cihazın/uç noktanın sahibi olduğu; belleği okuyup kodu değiştirebildiği ve hata ayıklayıcı bağlayabildiği saldırgan modelidir.	§1.1
RASP mimarisi	RASP denetimleri yükleme anında, hassas işlem öncesinde ve arka planda düzensiz aralıklarla, kodun birçok noktasında çalışır ve sonucu bir <code>if</code> 'e değil bir veri bağımlılığına bağlar.	§2
Bütünlük / self-hashing	Self-hashing, uygulamanın kendi kodunun/dosyasının HMAC özetini çalışma zamanında hesaplayıp önceden bilinen bir altın değerle karşılaştırarak yama/kurcalamayı yakalamasıdır.	§3
Hata ayıklayıcı (debugger) algılama	Anti-debug, çekirdeğin/işletim sisteminin zaten tuttuğu bilgileri (Linux'ta TracerPid, Windows'ta PEB) okuyarak sürece bir hata ayıklayıcının bağlı olup olmadığını anlamaya çalışmaktır.	§4
Emülatör/VM algılama	Ortam algılama, CPUID hipervizör biti, satıcı imzası ve zamanlama ölçümü gibi ipuçlarıyla uygulamanın bir sanal makinede/emülatörde mi, yoksa gerçek donanımda mı çalıştığını sezmeye çalışmaktır.	§5
Kanca (hook) / LD_PRELOAD algılama	Kanca algılama, kritik bir fonksiyonun hangi paylaşımlı nesneden (<code>.so</code>) çözüldüğünü <code>dlsym + dladdr</code> ile denetleyerek LD_PRELOAD/Frida gibi araçlarla değiştirilip değiştirilmediğini bulmaktır.	§6
Dinamik bellek koruması	Dinamik bellek koruması, çalışan programın belleğindeki hassas verinin okunmasını (döküm kapatma, kilitleme) ve sessizce değiştirilmesini (gölge kopya + özet) zorlaştıran önlemler bütünüdür.	§7
Kök (root) göstergesi	Root göstergesi, cihazın/sürecin güvenlik kısıtlarının kaldırılmış (root/jailbreak) ya da yükseltilmiş yetkiyle çalışıyor olduğuna işaret eden, tek başına kanıt sayılmayan sinyallerdir.	§8.1
Bileşen imza doğrulaması	Bileşen imza doğrulaması, dinamik yüklenen bir modülün/paketin yüklenmeden önce beklenen imza/özetini taşıyıp taşımadığını denetleyerek yeniden paketlemeyi (repackaging) yakalamaktır.	§8.2
Kontrol akışı sayacı	Kontrol akışı sayacı, güvenlik kontrol noktalarının doğru sırayla geçildiğini bir anahtar zincirine (<code>acc = HMAC(acc, aşama)</code>) bağlayarak tek bir kontrolün yamalanmasını işe yaramaz kılan tekniktir.	§9
Tepki politikası ve cihaz bağlama	Tepki politikası, bir tehdit algılandığında sırrı silme, decoy döndürme, cihaza/sürüme bağlama ve sunucuya bildirme gibi seçenekler arasından saldırgana en az bilgi verecek yanıtı seçmektir.	§10

Kavramlar birbirine nasıl bağlanır?

Yukarıdaki kavramlar rastgele bir liste değildir; her biri bir öncekinin üstüne oturur. Bir kavramı ilerideki bölümlerde unutursanız, aşağıdaki tablonun "önkoşul" sütununa dönüp önce onu tazeleyin.

Terim	Önkoşulu	Hangi bölümde derinleşir?
RASP (algıla/savun/caydır)	çalışma anı, MATE modeli	Bölüm 1
RASP mimarisi (ne zaman/nerede/nasıl)	RASP'in üç işlevi	Bölüm 2
Bütünlük / self-hashing	özet (hash), Hafta 3 HMAC	Bölüm 3
Hata ayıklayıcı (debugger) algılama	çalışma anı	Bölüm 4
Emülatör/VM algılama	debugger algılamanın "sinyal, kanıt değil" dersi	Bölüm 5
Kanca (hook) / LD_PRELOAD algılama	debugger ve emülatör algılamanın atlatılabilirliği	Bölüm 6
Dinamik bellek koruması	Hafta 1 bellek düzeni, debugger/kanca algılama	Bölüm 7
Kök (root) göstergesi	ayrıcılık (yetki) kavramı	Bölüm 8.1
Bileşen/paket imza doğrulaması	Hafta 3 HMAC/dijital imza	Bölüm 8.2
Kontrol akışı sayacı	bütünlük denetiminin "tek nokta" zaafı	Bölüm 9
Tepki politikası	bütün algılama sinyalleri	Bölüm 10
Cihaz bağlama	Hafta 3 HKDF, tepki politikası	Bölüm 10

Bu haftayı nasıl okumalı?

Bölümler art arda inşa edilir: 3 → 4 → 5 → 6 (dört bağımsız algılama ailesi: bütünlük, hata ayıklayıcı, ortam, kanca), 7 (bunların bellek üzerindeki karşılığı), 8 (kök ve bileşen imzası — iki ayrı ama benzer denetim), 9 (tek tek denetimleri **atlanamaz** kılma), 10 (hepsini bir tepki politikasında birleştirme). Bir bölümü atlarsanız, sonraki bölümdeki "Demo 2'deki gibi" ya da "Bölüm 3'teki fikirle aynı" cümleleri havada kalır; zor gelen bir bölümü ikinci kez okuyup ilerleyin.

1. RASP nedir? Algılama → savunma → caydırma

İlk beş hafta boyunca uygulamayı **statik** olarak sağlamlaştırdık: girdiyi doğruladık (Hafta 1, 4), veriyi şifreledik ([Hafta 3](#)), kodu gizledik ([Hafta 4](#), 5). Ama bütün bunlar bir varsayıma dayanıyordu: **program yazıldığı gibi çalışacak**. Bu hafta o varsayımı bırakıyoruz ve **çalışma anına (runtime)** — programın derlenip diskte durduğu an değil, fiilen **çalıştığı** an — bakıyoruz. Uygulama, çalışırken düşman bir ortamda olabilir: kullanıcı cihazın sahibidir, root almış, hata ayıklayıcı bağlamış, kod parçalarını yamalamış, fonksiyonları kancalamış olabilir.

WAF ile RASP farkı

“kurcalandım mı, beni kim izliyor, hangi cihazdayım?”

WAF

uygulamanın **ÖNÜNDE**
HTTP isteklerini süzer
uygulamanın içini bilmez
sunucu tarafı

RASP

uygulamanın **İÇİNDE**
kendi kodunu, belleğini,
ortamını izler
MATE saldırısına cevap

RASP, cihazın güvenilmez olduğu durumlar için tasarlanmıştır.

✎ Kısa tarihçe: uygulamanın kendini koruması

- **1980'ler** — crack / anti-crack kültürü: **anti-debug** ve kendini denetleyen kodun kökeni buraya dayanır.
- **2000'ler** — DRM, mobil bankacılık ve ödeme uygulamaları korumayı **uygulamanın içine** taşımak zorunda kalır (cihaz artık güvenilir değildir).
- **2012** — Gartner **RASP** (Runtime Application Self-Protection) terimini ortaya atar: koruma, uygulamanın **içinde** çalışır.
- **2010'lar–bugün** — **OWASP MASVS-RESILIENCE** bu beklentileri denetlenebilir gereksinimlere çevirir ([13. hafta](#)).

Kısacası RASP yeni bir fikir değil; **MATE saldırısına** verilen kurumsal cevabın adıdır.

RASP (Runtime Application Self-Protection — Çalışma Zamanı Uygulama Öz Koruması), uygulamanın **kendi içine** gömülen, **çalışma zamanında** kendini izleyen ve saldırıya tepki veren korumalardır. Uygulama kendi kendinin muhafızdır.

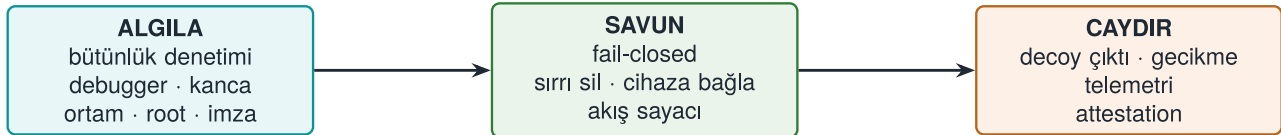
RASP dış korumalardan nasıl farklı?

Koruma	Nerede durur	Neyi görür
Ağ güvenlik duvarı	Ağ kenarında	Paketler
WAF (Web App Firewall)	Uygulamanın önünde	HTTP istekleri
RASP	Uygulamanın içinde	Kendi kodu, belleği, çalıştığı ortam
Bir WAF "dışarıdan gelen istek zararlı mı?" diye sorar. RASP " ben kurcalandım mı, beni kim izliyor,		
hangi cihazdayım? " diye sorar. Bu derste RASP'i mobil/gömülü ve masaüstü C/C++ bağlamında, uygulamanın kendini koruması		
olarak ele alıyoruz.		

RASP üç iş yapar; bu haftanın omurgası bu üçlüdür:

RASP döngüsü: algıla → savun → caydır

koruma uygulamanın İÇİNDE çalışır



Tek denetim değil, birbirini kontrol eden ÖRTÜŞEN AĞ → tek yama yetmez.

- **Algılama (detection):** Bir şeyin yanlış olduğunu anlamak — ikili yamalanmış, bir hata ayıklayıcı bağlı, ortam bir emülatör, uygulama root'lu cihazda, bileşen yeniden paketlenmiş.
- **Savunma (defense):** Algılamaya **anamlı** bir yanıt vermek — sırrı sil, işlemi reddet (fail-closed), anahtarı cihaza bağla, kritik işlemi kontrol akışına bağla.
- **Caydırma (deterrence):** Saldırganın işini **pahalılaştırmak** — çökmek yerine sahte (decoy) sonuç döndür, tepkiyi geciktir, olayı sunucuya bildir. Amaç kırılmazlık değil; saldırıyı **yeterince yavaşlatmaktır** (Hafta 4'teki "gizleme geciktirir, engellemez" ilkesinin RASP hâli).

1.1 Saldırgan modeli: MATE ("Man-At-The-End")

Hafta 1'de üç saldırı tipini görmüştük. RASP'in dünyası tümüyle **MATE**'tir: saldırı uygulamanın **çalıştığı uç noktanın sahibidir**. Sunucuya karşı saldırı dışarıdadır (ve TLS, kimlik doğrulama korur); ama istemci uygulamada saldırı **her şeyi görür**: belleği okuyabilir, kodu değiştirebilir, hata ayıklayıcıyla adım adım izleyebilir. Bu, **Hafta 11'deki beyaz kutu** modeliyle aynı varsayımdır.

MATE saldırı modelini, Hafta 1'de gördüğümüz diğer saldırı tiplerinden ayıran şey **konumdur**, yetenek değil:

Saldırgan tipi	Nerede durur	Neyi görebilir	Klasik savunma
Ağ saldırısı	İstemci ile sunucu arasında	Yalnız şifreli trafik (TLS varsa)	TLS, kimlik doğrulama
Sunucu saldırısı	Sunucunun dışında	Yalnız gönderdiği istekler ve aldığı yanıtlar	Girdi doğrulama, yetkilendirme, hız sınırlama
MATE (uçtaki saldırı)	Uygulamanın çalıştığı cihazın içinde , sahibi	Belleği, ikili dosyayı, çalışma anındaki her adımı	RASP: bütünlük, anti-debug, kanca algılama, cihaz bağlama

Bu tablo neden önemli? Çünkü bir MATE saldırısına karşı **ağ saldırıları savunmaları** (TLS, sunucu tarafı girdi doğrulama) hiç işe yaramaz — TLS bağlantısının **içeriğini** korur, ama saldırı bağlantıyı açan uygulamanın **kendisidir**; şifreyi çözdüğü anda veriyi kendi belleğinde açık olarak görür. Bu yüzden RASP, önceki haftaların savunmalarının **yerine değil, üstüne** eklenir.



RASP'in acı gerçeği

Cihazın sahibi saldırırsa, **hiçbir istemci tarafı koruma nihai olarak kırılmaz değildir**. Yeterli zaman ve beceriyle her kontrol atlatılabilir. O hâlde RASP neden var? Çünkü amaç **kırılmazlık değil**, saldırıyı **ölçeklenemez ve pahalı** kılmaktır: bir cihazı kırmak günler alıyorsa ve kırık her cihazda tekrar gerekiyor **ve** sunucu tarafı anormallikleri yakalıyor, saldırı ekonomik olarak çökebilir. RASP **derinlemesine savunmanın** çalışma zamanı katmanıdır; **tek başına değil**, kriptoloji (Hafta 3, 10), gizleme (Hafta 4, 9), whitebox (Hafta 11) ve **sunucu tarafı denetimle** birlikte anlam kazanır.

1.2 Bu haftanın örnek mimarisi ve ana fikri

Dönem boyunca kullandığımız örnek mimariyi hatırlayın: bir **mobil ödeme uygulaması** ve onun kritik işlerini yapan bir **güvenlik kütüphanesi** (native C/C++). Telefon **güvenilmez ortamdır**. Bu hafta o kütüphaneye, çalışırken kendini koruyan bir **RASP katmanı** ekliyoruz. Bütün örnek değerler **sentetiktir** (uydurma); gerçek anahtar, kart numarası, cihaz kimliği ya da ürün adı kullanılmaz.

Ana fikir tek cümlede: **tek bir kontrole güvenme**. Her algılama sinyali tek başına atlatılabilir; güç, **birden çok bağımsız sinyalin**, bir **tepki politikasının** ve **cihaz bağlamanın** birlikte çalışmasından gelir. Bu, Hafta 3'teki **güvenlik kabuğunun** çalışma zamanındaki karşılığıdır.

1.3 İşlenmiş örnek: aynı olayı üç farklı şekilde ele almak

Algılama, savunma ve caydırmanın neden **ayrı** kavramlar olduğunu somutlaştıralım. Senaryo: kriptoloji kütüphanesi, ödeme anahtarını açmadan hemen önce bir hata ayıklayıcının bağlı olduğunu fark ediyor (Bölüm 4'teki TracerPid sinyali > 0). Üç uygulamacı bunu üç farklı şekilde ele alıyor:

1. **Yalnız algılama, tepki yok:** Kod `supheli` değişkenini artırır ama hiçbir yere bakmaz; fonksiyon normal akışına devam eder. Sonuç: hata ayıklayıcı bağlı bir saldırgan anahtarı **doğrudan okur**. Algılama boşa gitmiştir — çünkü bir savunmaya bağlanmamıştır.
2. **Algılama + kaba savunma:** Kod `if (supheli) exit(1);` yazar. Sonuç: saldırgan anahtarı okuyamaz, ama programın **tam olarak hangi satırda** durduğunu görür; birkaç deneme ile o `if`'i yamalar (Bölüm 3'teki "checker'ın kendisi de yamalanır" dersiyle aynı zaaf).
3. **Algılama + savunma + caydırma:** Kod, sinyali doğrudan `exit()` 'e değil, anahtar türetme işlemine **veri olarak** karıştırır (Bölüm 9'daki kontrol akışı sayacının küçük hâli) ve tetiklenirse **decoy** bir sonuç üretir (Bölüm 10). Sonuç: saldırgan bir hata mesajı görmez; "başarılı" görünen ama işe yaramaz bir sonuç alır ve hangi kontrolün tetiklendiğini bilemez.

Bu üç seçenek aynı **algılama** koduna sahiptir; farkları yalnız **savunma** ve **caydırma** katmanındadır. Bu haftanın demoları ilerledikçe hep üçüncü yaklaşıma yaklaşıcağız (bkz. Bölüm 10.2'deki tam karşılaştırma).

2. RASP mimarisi: denetimler ne zaman, nerede ve nasıl çalışır?

Önümüzdeki bölümlerde tek tek denetimleri (bütünlük, hata ayıklayıcı, ortam, kanca, root) göreceğiz. Ama sahada bir RASP çözümünün gücü, tek tek denetimlerin ne kadar akıllı olduğundan çok, bu denetimlerin **nasıl bir araya getirildiğinden** gelir. Tek bir yerde, tek bir kez çalışan, sonucu tek bir `if` ile değerlendirilen bir denetim, ne kadar iyi yazılmış olursa olsun, bulunup atlatılması en kolay denetimdir. Bu bölüm, denetimleri yerleştirmenin tasarım ilkelerini anlatır.

Ne zaman çalışır? Üç tetikleme anı

An	Örnek	Neden?
Yükleme anı	Kütüphane belleğe yüklendiğinde (Android'de native kütüphanenin yükleme fonksiyonu)	Ortam baştan güvenilmezse hiçbir sır açılmaz
Hassas işlem öncesi	Anahtar çözmeden, ödeme hesaplamadan, sunucuya kimlik doğrulamadan hemen önce	Saldırgan uygulama açıldıktan sonra hata ayıklayıcı bağlayabilir
Arka planda, düzensiz aralıklarla	Ayrı bir iş parçacığında, rastgele zamanlarda	Sabit zamanlı denetim kolayca tahmin edilir ve "arasına" girilir

Tek bir tetikleme anı yetmez: yalnız açılışta yapılan bir hata ayıklayıcı denetimini, uygulama açıldıktan sonra bağlanan bir hata ayıklayıcı hiç görmez.

Nerede çalışır? Dağıtık denetim

- **Çok noktalılık:** Aynı denetim tek bir fonksiyonda değil, kodun **birçok yerinde**, her seferinde biraz farklı biçimde yapılır. Saldırgan bir noktayı yamaladığında diğerleri hâlâ çalışır.
- **Karşılıklı denetim:** Yönetilen katman (Java) ile native katman birbirinin bütünlüğünü denetler; biri değiştirilirse diğeri fark eder ([5. haftadaki](#) ayırık parmak izi ve [1. haftadaki](#) arayüz tablosunda "karşılıklı bütünlük denetimi").
- **Korunan fonksiyonun içinde:** Değerli bir fonksiyon (ör. dize çözme ya da anahtar türetme), kendi giriş noktasında birkaç denetimi **kendisi** yapar. Sahada böyle tek bir fonksiyonun, yüklenme anında zaten yapılmış denetimlere ek olarak kendi başına on beşe yakın denetimle (kod bütünlüğü, hata ayıklayıcı, ortam, root, kontrol akışı sayacı)

korunduğu görülür. Performans nedeniyle, yalnız iç kullanımdaki sürümlerinde bu denetimlerin bir kısmı giriş noktasına indirilir.

- **Sahte parametreler:** Hassas fonksiyonların giriş noktalarında, programcının okunabilirliği için makroyla gizlenen ve bir bütünlük denetimini tetikleyen ek parametreler bulunabilir (4. haftadaki "sahte parametreler").

Nasıl karar verir? Merkezî değil, veri bağımlı

En zayıf tasarım şudur:

Zayıf: tek karar noktası

```
if (hata_ayiklayici_var() || root_var() || butunluk_bozuk())
    return HATA;
/* bu tek satırı değiştiren her şeyi atlatır */
anahtari_coz();
```

Tek bir karşılaştırmayı ters çeviren (ya da `hata_ayiklayici_var` fonksiyonunun hep "hayır" döndürmesini sağlayan) saldırgan bütün korumayı geçer. Güçlü tasarımda denetimlerin sonucu bir `if` 'e değil, **sonraki hesabın verisine** karışır:

- Denetim sonuçları anahtar türetmeye katılır: ortam temizse doğru anahtar, değilse **farklı ama geçerli görünen** bir anahtar çıkar (kontrol akışı sayacı ve tepki politikasındaki decoy çıktı bu fikrin iki uygulamasıdır).
- Denetim sonuçları opak değerlerle taşınır (4. hafta): tek bir baytı değiştirerek "geçti"ye çevrilemez.
- Tepki, tetikleyiciden **zaman ve kod olarak uzak** bir yerde verilir (tepki politikası bölümü).

"Veri bağımlılığı" ne demek, somut olarak? Zayıf tasarımda `if (kontrol_gecti) anahtari_kullan();` yazılır — `anahtar` zaten bellekte hazır, `if` yalnız onu **kullanıp kullanmayacağına** karar verir; saldırgan bu `if` 'i yamalarsa anahtar **zaten oradadır** ve kullanılır. Güçlü tasarımda ise anahtarın kendisi **henüz yoktur**; kontrol sonucundan **hesaplanarak üretilir**: `anahtar = HMAC(kontrol_sonucu, sabit_tuz)`. Kontrol başarısızsa `kontrol_sonucu` farklı bir değerdir, `anahtar` da farklı (ve işe yaramaz) çıkar — saldırganın atlayacağı bir `if` **yoktur**, çünkü ortada atlanacak bir karar değil, hesaplanacak bir değer vardır. Bölüm 9.1'deki `acc` zinciri ve Bölüm 10.1'deki cihaz anahtarı, bu fikrin iki somut uygulamasıdır.

Katmanlar birbirini korur

Katmanlar birbirini korur

aynı hassas işlemi saran katmanlar

Gizleme (4, 5, 9. hafta)

denetimleri BULMAYI zorlaştırır

Bütünlük denetimi

denetimlerin YAMALANMASINI fark eder

Debugger / kanca / ortam

çalışma anı ANALİZİNİ fark eder

Kontrol akışı sayacı

denetimin ATLANMASINI fark eder

HASSAS İŞLEM

korunan çekirdek

Saldırganın hepsini aynı anda aşması gerekir; bu da saldırı potansiyelini yükseltir.

Her katman bir öncekinin zayıflığını kapatır: gizleme denetimleri saklar, bütünlük denetimi gizlenmiş denetimlerin **değiştirilmesini** fark eder, kontrol akışı sayacı bütünlük denetiminin **atlanmasını** fark eder, cihaz bağlama bütün bunlar aşılsa bile verinin başka bir yerde **kullanılmasını** engeller; sonucu doğrulaması ise istemci tamamen ele geçirilse bile son sözü söyler (tepki politikasındaki telemetri ve sonucu tarafı risk motoru).

2.1 İşlenmiş örnek: tek bir fonksiyonu üç noktada korumak

"Çok noktalılık" soyut kalsın. Örnek mimarideki `anahtar_coz()` fonksiyonunu (ödeme anahtarını açan native fonksiyon) ele alalım ve üç tetikleme anını sırayla uygulayalım:

- Yükleme anı:** Kütüphane belleğe yüklendiğinde (native kütüphanenin başlatma fonksiyonunda) bir kez bütünlük denetimi (Bölüm 3) çalışır. Ortam en baştan bozursa, `anahtar_coz` hiçbir zaman çağrılmaz hale getirilir (bir global bayrak sıfırlanır).
- Hassas işlem öncesi:** `anahtar_coz()` 'un kendi girişinde, ayrıca bir anti-debug denetimi (Bölüm 4) ve bir kanca denetimi (Bölüm 6) çalışır — çünkü saldırgan hata ayıklayıcıyı **yükleme anından sonra** bağlamış olabilir.
- Arka planda, düzensiz aralıklarla:** Ayrı bir iş parçacığı, rastgele aralıklarla (ör. 2–7 saniye) ortam denetimini (Bölüm 5) tekrar çalıştırır; bir sonraki `anahtar_coz()` çağrısı bu iş parçacığının bulduğu sonucu okur.

Sonuç: `anahtar_coz()` 'u atlatmak isteyen saldırganın **üç ayrı zaman noktasındaki üç ayrı denetimi** aynı anda yenmesi gerekir. Yalnız 2. noktayı (fonksiyonun kendi girişini) yamalamak, 1. ve 3. noktadaki denetimleri etkilemez — saldırgan işini bitirdiğini sanır ama arka plan iş parçacığı birkaç saniye içinde tutarsızlığı yakalar ve tepki politikası devreye girer (Bölüm 10).



Sahada nasıl uygulanır?

Kart şemalarının uygulama koruma gereksinimleri tam bu katmanları sayar: yetkisiz değişikliğe karşı koruma, çalışma anında öz bütünlük denetimi, desteklenmeyen cihazda çalışmama, sürüm geri almayı engelleme, statik ve dinamik tersine mühendisliğe ve kanca tekniklerine karşı savunma, hata ayıklama ve test ortamını algılama, uzlaşma durumunda arka uca ve kullanıcıya bildirim ve verinin silinmesi, işletim sistemi korumalarının açık olduğunun denetlenmesi. Sertifikalı bir kütüphanenin kılavuzunda native tarafta on dört, Java tarafında on dört ayrı RASP önlemi bu gereksinimlere tek tek eşlenir.

3. Bütünlük denetimi: uygulama kendini doğrular (self-hashing)

RASP'in ilk ve en temel adımı: **"Ben değiştirildim mi?"** Saldırgan bir lisans kontrolünü, bir `if (odendi)` dalını ya da bir kriptografi çağrısını **yamalayarak** (patching) korumayı atlatmaya çalışır ([Hafta 4](#)'te gördüğümüz `je → jmp` yaması). Savunma: uygulama, kendi ikili dosyasının (ya da korunan bir kod/veri bölgesinin) bir **özetini** çalışma zamanında hesaplar ve önceden bilinen **altın (golden)** değerle karşılaştırır. Tek bir bayt bile değişmişse, yama yakalanır.

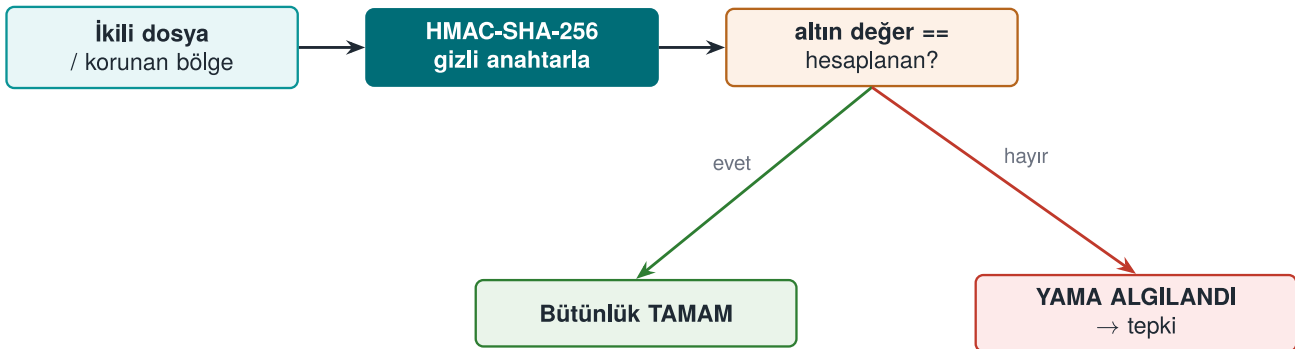


Kitap tarifi ve güncelleme

Ders kitabı (Viega & Messier) bunu **tarif 12.2 (Detecting Modification)**'te **CRC32** ile yapar. CRC32 **kriptografik değildir**: saldırgan kodu yamalıyıp aynı CRC'yi verecek şekilde kolayca ayarlayabilir. Biz bunu **HMAC-SHA-256** ile güncelliyoruz (bkz. [Hafta 3, §2.4](#)): doğru HMAC'i üretmek için gizli anahtar gerekir. Yazarın CRC seçmesinin bir nedeni vardır: asıl saldırı zaten **checksum kodunun kendisini** yamalamaktır — bu bölümün ana tartışması budur.

Bütünlük denetimi (self-hashing)

uygulama kendi baytlarını doğrular



Tek başına yetmez: denetimin kendisi de yamalanabilir → örtüşen denetim gerekir.

Demo 1 — Çalışma zamanı bütünlük denetimi ve yama tespiti

Demo 1 · `code/week-06/01-butunluk-hmac` · self-hashing, HMAC-SHA-256 (Tarif 12.2 güncellemesi)

Program kendi ikili dosyasının (`oz`) HMAC-SHA-256 özetini hesaplayıp "altın" değer olarak kaydeder, sonra doğrular. Ardından ikili dosyanın bir **kopyası** alınıp bir baytı değiştirilir (yama simülasyonu, demonun kendi `cikti/` klasöründe) ve yamalı kopyanın özeti altın değerle karşılaştırılır.

Özün kodu (dosyayı okuyup HMAC hesaplar; `cen429_kripto.h`'dan `kripto_hmac_sha256`):

butunluk.c (özet)

```
/* Kendi yolunu bul: Linux /proc/self/exe, Windows GetModuleFileNameA */
oz_yol(yol, sizeof(yol));
dosya_oku(yol, &veri, &boy);
kripto_hmac_sha256(RASP_ANAHTAR, 32, veri, boy, ozet); /* HMAC over the file */

/* dogrula: sabit zamanli karsilastirma (zamanlama sızıntısını onler) */
unsigned char fark = 0;
for (int i = 0; i < 32; i++) fark |= beklenen[i] ^ ozet[i];
if (fark == 0) { /* BUTUNLUK TAMAM */ } else { /* YAMA ALGILANDI */ }
```

WSL'de alınan **gerçek çıktı** (kısaltılmış):

sh demo.sh — çıktı

```
ADIM 2 - Normal calisma: butunluk dogrulanir (yama yok)
Beklenen   : bae498a6a982279a833bf7e33d2db9cf...ee4d01e6
Hesaplanan : bae498a6a982279a833bf7e33d2db9cf...ee4d01e6
SONUC: BUTUNLUK TAMAM - hedef degismemis.

ADIM 3 - Saldiri: ikili dosyanin bir KOPYASI alinip 1 bayti degistiriliyor
> ofset 21268 baytini XOR 0xFF ile degistir
ADIM 4 - Yamali kopyanın HMAC'i altin degerle karsilastiriliyor
Beklenen   : bae498a6a982279a833bf7e33d2db9cf...ee4d01e6
Hesaplanan : 05fb42748cb5b57305f3ffacd61ad757...131c6705
SONUC: YAMA ALGILANDI - hedef degistirilmis! (tamper) (cikis kodu 3)
```

Yama yokken özet altın değerle **birebir** aynı çıktı. Tek bir baytı değiştirince özet tamamen değişti (çığ etkisi) ve yama yakalandı. Karşılaştırma **sabit zamanlı** yapılır (`memcmp` değil), çünkü [Hafta 3'te](#) gördüğümüz gibi düz `memcmp` ilk farklı baytta durup zamanlama sızdırır.

3.1 İşlenmiş örnek: HMAC hangi baytları özetliyor, yama onu nasıl bozuyor?

Hangi baytlar özetleniyor? `hedef_hmac()` fonksiyonuna bakınca cevap nettir: `dosya_oku()` hedefin (kendi ikili dosyanın) **tamamını** belleğe okur (`fread` ile ofset 0'dan dosya sonuna kadar) ve `kripto_hmac_sha256` bu **bütün bayt dizisini** tek seferde özetler — belirli bir fonksiyon ya da bölüm değil, dosyanın **her baytı**. `demo.sh`'in gerçek çıktısında `ofset 21268` değiştiriliyordu; bu, dosyanın **ortasına** (`BOY / 2`) denk gelir — demek ki o çalıştırmada ikili dosyanın toplam boyutu yaklaşık $21268 \times 2 \approx 42536$ bayttı. Saldırgan dosyanın hangi baytını değiştirirse değiştirsin (başı, ortası, sonu fark etmez), o bayt hesaba dahildir ve özet değişir.

Yama onu nasıl bozuyor — küçük bir örnekte adım adım. Aynı çığ etkisini, koca bir ikili dosya yerine **tek bir karakterlik** bir metinde görelim; mantık birebir aynıdır. "MERHABA" dizesini sabit bir anahtarla HMAC-SHA-256'layalım, sonra son karakteri `A`'dan `B`'ye çevirip tekrar hesaplayalım:

Tek karakterlik deęisikligin HMAC'e etkisi (gercek openssl ciktisi)

```
printf 'MERHABA' | openssl dgst -sha256 -hmac "ornek-anahtar"
printf 'MERHABB' | openssl dgst -sha256 -hmac "ornek-anahtar"
```

```
SHA2-256(stdin)= 4d45a6382deee7f75c75b3cf5a6a81a2776d317e1db9ec9b0125b69e09492615
SHA2-256(stdin)= 4fbdf8768822083c3fc3797c5f1d278dd1ea58ab0bfde31a8ebf85e260fbcdb7
```

Girdinin **7 baytından yalnız 1 tanesi** (son karakter, A =0x41 → B =0x42; bu iki deęer yalnız **iki bitte** ayrılır) deęiřti; ama ıktının **32 baytının tamamı** farklı. Bu, kriptografik zet fonksiyonlarının **ıę etkisi**dir: girdide tek bir bit deęiřirse, ıktının ortalama **yarısı** deęiřir. Demo 1'in gercek ıktısında grdęnz bae498a6...ee4d01e6 → 05fb4274...131c6705 dnřm de aynı zellięin, koca bir ikili dosya zerindeki hlidir: XOR 0xFF tek bir baytın **her bitini** ters evirir, bu da HMAC-SHA-256'nın i iřleyiři (SHA-256'nın sıkıřtırma fonksiyonunun her turu) yoluyla btn 256 bit ıktıya yayılır.

Neden bu, denetimin iře yaraması iin gerekli?

ıę etkisi olmasaydı (r. CRC32'de olduęu gibi zayıf bir zet fonksiyonunda), saldırgan yamaladıęı baytın yanına **telafi edici** birkaç bayt daha ekleyerek zeti eski deęerine geri getirebilirdi (checksum'ı "dzeltme" saldırısı). HMAC-SHA-256 gibi kriptografik bir zette bu, anahtarı bilmeden **pratik olarak imknsızdır** — bu yzden Blm 3'n bařındaki "Kitap tarifi ve gncelleme" notunda CRC32'nin neden HMAC'e ykseltildięini grmřtk.

Btnlk denetimi tek bařına neden yeterli deęil?

HMAC anahtarı **ikili dosyanın iindedir**. Yeterince kararlı bir saldırgan: 1. Denetleyici (checker) fonksiyonun **kendisini** yamalayarak "her zaman TAMAM dndr" yapabilir. 2. Yamayı yapıp **altın deęeri de** yeniden hesaplayabilir (anahtarı bulursa).

Bu yzden gercek RASP btnlk denetimini řu yle glendirir: **(1) oklu/rtřen denetleyiciler** (bir aę hlinde, biri dięerini de kontrol eder), **(2) sonucu bir veri baęımlılıęı yapmak** — zeti doęrudan bir anahtar/atlama tablosu indeksi olarak kullanmak, bylece yama davranıřı bozar (bkz. Blm 9, Demo 5), **(3) sonucu tarafı attestation** — istemcinin btnlk kanıtını sunucuya nonce ile gndermek.

Kural

Kritik kod/veri blgelerinin btnlęn alıřma zamanında **kriptografik** bir zetle (HMAC/imza) doęrulayın, CRC ile deęil. Denetimi **tek noktada ve yalnız bařlangıta** yapmayın; periyodik, oklu ve rtřen yapın. Sonucu mmknse bir **veri baęımlılıęına** baęlayın.

Sahada nasıl uygulanır? / Deęerlendirici nasıl test eder?

rnek mimaride native ktphane, hem kendi .so / .dll 'inin hem de st katmanın (r. Java/DEX) birleřik zetini doęrular; **karřılıklı** (mutual) doęrulama yapar (Katalog K6, Demo 6). Deęerlendirici ikili dosyayı yamalayıp algılamının tetiklenip tetiklenmedięine, ardından **denetleyici kodun kendisini** yamalayarak korumayı atlatıp atlatamadıęına bakar (tek denetleyici mi, aę mı?).



Sık hatalar ve kontrol listesi

- ✓ Bütünlük **kriptografik** mi (CRC değil)?
- ✓ Denetim yalnız başlangıçta değil, **periyodik** mi?
- ✓ Tek bir denetleyici mi var, yoksa **örtüşen** denetleyiciler mi?
- ✓ Sonuç bir davranışa (veri bağımlılığı) mı bağlı, yoksa yalnız bir **if** mi?
- ✓ Sunucu tarafı bir **attestation** var mı (yalnız istemciye güvenmeme)?

"Periyodik" ne sıklıkta olmalı? Bir maliyet-gecikme dengesi. Bölüm 2'deki "arka planda, düzensiz aralıklarla" ilkesini bir sayıyla somutlaştıralım. Denetim aralığı ne kadar **kısa** olursa, bir yamanın fark edilmesi o kadar **hızlı** olur ama CPU/batarya maliyeti de o kadar **artar**; aralık ne kadar **uzun** olursa saldırgan o kadar fazla "fark edilmeden çalışma" penceresi kalır:

Denetim aralığı	Bir yamanın fark edilme gecikmesi (üst sınır)	Tipik ek CPU yükü
Yalnız açılışta (1 kez)	Sınırsız (hiç fark edilmeyebilir)	En düşük
Her 30 saniyede bir	~30 saniye	Düşük
Her 2–7 saniyede bir (rastgele)	~7 saniye, tahmin edilemez anda	Orta
Her kritik işlemten hemen önce (Bölüm 2'deki 2. nokta)	Neredeyse anlık	Yalnız kritik işlemlerde

Sahada tercih edilen, bu dört seçeneğin **hiçbiri tek başına değil, birlikte** kullanılmasıdır: açılışta bir kez (ucuz, geniş kapsamlı), kritik işlem öncesi her seferinde (ucuz, dar kapsamlı ama en kritik an), ve arka planda rastgele aralıklarla (ucuz değil ama saldırganın "denetim ne zaman çalışıyor" diye tahmin etmesini engeller). Sabit bir aralık (ör. tam olarak her 30 saniyede bir) saldırganın bir **tahmin edilebilirlik** sunar; rastgele aralık bunu engeller — bu yüzden tablodaki üçüncü satır "rastgele" olarak işaretlidir.

4. Hata ayıklayıcı (debugger) algılama

Saldırganın en güçlü aracı bir **hata ayıklayıcıdır** (gdb, lldb, x64dbg, WinDbg, Visual Studio): programı durdurur, adım adım izler, belleği ve anahtarları okur, dalları değiştirir. RASP, bir hata ayıklayıcının **bağlı olup olmadığını** anlamaya çalışır.

Zamanlama tabanlı sezme

debugger kodu yavaşlatır



Yanlış pozitif riski: yavaş cihaz, arka plan yükü. Eşiği ölçerek seçin.

Hata ayıklayıcı algılama yolları

hiçbiri tek başına kesin değil — sinyaller birlikte tartılır

İşletim sistemi bayrağı

Linux /proc/self/status TracerPid · Windows IsDebuggerPresent

PEB / NtGlobalFlag

Windows süreç yapısındaki izler

Zamanlama

iki nokta arası süre eşiği aşıyor mu?

Kendi kendini izleme

ptrace(PTRACE_TRACEME) başarısızsa biri izliyor

Her sinyal atlatılabilir; güç, birden çok bağımsız sinyalin birlikteliğinden gelir.

Ortam	Gösterge	Kitap
Linux / WSL	/proc/self/status → TracerPid (>0 ise bir süreç bizi ptrace ile izliyor); ana sürecin adı gdb/strace/ltrace mı	Tarif 12.13
Windows	IsDebuggerPresent (PEB BeingDebugged); CheckRemoteDebuggerPresent; PEB NtGlobalFlag (0x70 bitleri)	Tarif 12.14

i 2003'ten bugüne: neler eskidi?

- **SoftICE (Tarif 12.15) tamamen öldü.** Yerine modern hipervizör/ring-0 hata ayıklayıcıları, x64dbg, WinDbg, gdb, lldb ve özellikle **Frida/DBI** (dinamik ikili enstrümantasyon) geçti (bkz. Bölüm 6).
- `IsDebuggerPresent` (12.14) hâlâ var ama **tek başına zayıf**; yanına `PEB BeingDebugged` / `NtGlobalFlag`, `CheckRemoteDebuggerPresent`, timing tabanlı tespit eklenir.
- `ptrace` tekniği (12.13) hâlâ geçerli; modern eklemeler: `/proc/self/status` **TracerPid** çapraz kontrolü.

Demo 2 — Hata ayıklayıcı algılama (Linux ptrace/TracerPid, Windows PEB)

i Demo 2 · `code/week-06/02-hata-ayiklayici` · anti-debug, yalnız okuma

Program birden çok bağımsız sinyale bakar ve **hiçbir şeyi değiştirmez** (kendini `ptrace` ile de izlemez; yalnız `/proc` ve PEB alanlarını okur). `demo.sh` programı önce normal, sonra `gdb` altında çalıştırır.

antidebug.c (Linux özeti)

```
/* /proc/self/status icinden TracerPid oku; >0 ise izleniyoruz */
long tp = tracer_pid();
if (tp > 0) supheli++; /* bir surec bizi ptrace ile izliyor */
```

sh demo.sh — gerçek çıktı (WSL)

ADIM 1 - Normal calisma (hata ayiklayici YOK): temiz beklenir
 1) `/proc/self/status TracerPid` : 0 (izleyen yok)
 2) Ana surec (parent) adi : sh
 SONUC: Temiz - izleyen bir arac gorunmuyor.

ADIM 2 - gdb altinda calistir (TracerPid > 0 beklenir)
 1) `/proc/self/status TracerPid` : 2909 (IZLENIYOR)
 SONUC: Hata ayiklayici/izleme ARACI algilandi (2 sinyal).

Hata ayıklayıcı yokken TracerPid **0**, `gdb` altında **>0** çıktı; program iki durumu ayırt etti. Windows'ta Visual Studio ile **F5** (hata ayıklayıcıda) çalıştırırsanız `IsDebuggerPresent` **EVET** döndürür ve sonuç "algılandı" olur.

4.1 İşlenmiş örnek: TracerPid'in 0'dan sıfır olmayana geçişini adım adım izlemek

Demo 2'nin çıktısında TracerPid'in 0'dan 2909'a geçtiğini gördük; şimdi bu geçişin **neden** olduğunu adım adım açalım. `/proc/self/status`, çekirdeğin her süreç için tuttuğu, o sürecin kendisi tarafından da okunabilen bir metin dosyasıdır (Hafta 1'de `/proc` dosya sistemini "çekirdeğin sürece açtığı bir pencere" olarak tanımlamıştık). İçindeki `TracerPid:` satırı, o süreci `ptrace()` sistem çağrısıyla izleyen sürecin **PID**'ini (Process ID — süreç kimliği) tutar; kimse izlemiyorsa değer **0**'dır.

Adım 1 — hata ayıklayıcı yokken. `antidebug` programı doğrudan kabuktan çalıştırılır ve kendi `/proc/self/status` dosyasını okur (temsili, kısaltılmış içerik):

```
/proc/self/status (hata ayiklayici YOK, temsili)
```

```
Name:      antidebug
Pid:       4821
PPid:      4820      <- calistiran kabuk (sh)
TracerPid: 0        <- kimse ptrace ile izlemiyor
```

Program bu satırı okur, sayısal değeri 0 bulur; `tp > 0` koşulu **yanlış** olduğu için `supheli` değişkenini artırmaz. Demo çıktısındaki "SONUC: Temiz" satırı buradan gelir.

Adım 2 — gdb altında çalıştırılınca. `demo.sh` ikinci geçişte programı `gdb` içinden başlatır. `gdb`, hedef süreci başlatırken ya da ona bağlanırken çekirdeğe "bu süreci ben izliyorum" der; bu, `ptrace(PTRACE_TRACEME, ...)` ya da `ptrace(PTRACE_ATTACH, ...)` çağrısıyla olur. Çekirdek bunu **kaydeder** ve izlenen sürecin `/proc/self/status` dosyasındaki `TracerPid` alanını, izleyen `gdb` sürecinin **kendi PID'yle** doldurur:

```
/proc/self/status (gdb ALTINDA, temsili)

Name:      antidebug
Pid:       4821
PPid:      2909      <- artık ana surec gdb
TracerPid: 2909      <- gdb bizi ptrace ile izliyor
```

Bu, tam olarak demo çıktısında görülen 2909 değeridir. Programın kendisi hiçbir şey **yapmamıştır** — yalnız çekirdeğin zaten tuttuğu bir alanı **okumuştur**; bu yüzden teknik "yalnız okuma" (read-only) olarak sınıflandırılır ve hiçbir yan etkisi yoktur (demonun `README` 'sindeki güvenlik sözü tam bu nedenle tutulabilir).

Adım 3 — karar. `tp > 0` olduğu için `supheli` sayacı 1 artar; ana sürecin adı da `gdb` olduğu için ikinci bağımsız bir sinyal (`ana_surec_adi` denetimi) daha eklenir; demo çıktısındaki "algılandı (2 sinyal)" ifadesi bu ikisinin toplamıdır. Tek bir alanın okunması yeterli olmasa da, çekirdek düzeyinde tutulan bu bilgi **sahtelemesi zor** bir sinyaldir: `TracerPid` 'i doğrudan sıfırlamak sürecin kendi belleğinden değil çekirdek veri yapısından okunduğu için mümkün değildir — saldırganın bunu gizlemesi ancak `/proc/self/status` 'u okuyan **kütüphane fonksiyonuna** kanca takmasıyla (hooking) mümkündür, ki bu da tam olarak Bölüm 6'nın konusudur.

4.2 Windows karşılığı: `NtGlobalFlag` 'teki bitleri okumak

Windows'ta doğrudan bir `/proc` dosyası yoktur, ama aynı fikrin (çekirdeğin zaten tuttuğu bir bilgiyi okumak) bir karşılığı vardır: **PEB** (Process Environment Block — her sürecin çekirdek tarafından tutulan, sürecin kendisinin de okuyabildiği bir bellek yapısı). PEB'in `NtGlobalFlag` alanı, süreç bir hata ayıklayıcı altında **başlatıldıysa** (yalnız sonradan bağlanmadıysa) üç özel bayrak biti taşır:

Bit (onaltılık)	Anlamı
0x10	<code>FLG_HEAP_ENABLE_TAIL_CHECK</code> — yığın sonu denetimi
0x20	<code>FLG_HEAP_ENABLE_FREE_CHECK</code> — serbest bırakılan yığın denetimi
0x40	<code>FLG_HEAP_VALIDATE_PARAMETERS</code> — yığın parametre doğrulama

Bu üç bit **birlikte** `0x70` 'i oluşturur (`0x10 | 0x20 | 0x40 = 0x70`); Windows yükleyicisi, süreci bir hata ayıklayıcı altında başlatırken yığın ayırıcısını daha sıkı denetimli bir moda **otomatik olarak** geçirir ve bu üç biti işaretler. Demo 2'nin Windows kodundaki `(ntglobal & 0x70) != 0` satırı tam bunu okur: `NtGlobalFlag` değerinin bu üç bitten **en az birini** taşıyıp taşımadığına bakar. Tıpkı `TracerPid` gibi, bu da yalnız **okuma** işlemidir; program hiçbir şeyi değiştirmez, çekirdeğin/yükleyicinin zaten yazdığı bir alanı okur.

” Tespit–karşı-tespit yarışı

Anti-debug klasik bir **silah yarışıdır**. Her tespit tekniğinin bir karşı-tekniki vardır: saldırgan `ptrace`'i `LD_PRELOAD` ile no-op yapabilir (Demo 4), `IsDebuggerPresent` 'in dönüşünü yamalayabilir, PEB bayrağını temizleyebilir. Bu yüzden **tek bir kontrole asla güvenilmez**; sinyaller çoğaltılır ve bir tepki politikasına bağlanır (Demo 8).

✓ Kural

Anti-debug'ı **tek bir** `if (IsDebuggerPresent()) exit();` olarak yazmayın (saldırgan o çağırışı yamalar). Birden çok bağımsız sinyal (`ptrace` + `TracerPid` + `timing` + `parent`) toplayın, sonucu doğrudan bir davranışa/anahtara bağlayın ve tepkiyi **geciktirin** (hemen çıkmayın; saldırganın hangi kontrolün tetiklediğini anlamasını zorlaştırın).



Sık hatalar ve kontrol listesi

- ✓ Tek bir anti-debug çağrısına mı bağlısınız, birden çok sinyale mi?
- ✓ Tespit sonucu bir davranışa/anahtara mı bağlı, yoksa yalnız `exit()` mi?
- ✓ `ptrace(PTRACE_TRACEME)` gibi **yan etkili** bir teknik yerine salt okuma (`TracerPid`) mı tercih edildi?
- ✓ Tepki hemen mi, yoksa geciktirilmiş/örtük mü?

5. Ortam algılama: sanal makine ve emülatör

Saldırgan uygulamayı çoğu zaman kontrollü bir **analiz ortamında** — bir sanal makine, emülatör ya da kum havuzunda (sandbox) — çalıştırır. **Emülatör**, bir cihazı yazılımla **taklit eden** ortamdır (Android emülatörü, QEMU); **sanal makine (VM)** ise gerçek donanımı soyutlayıp üzerinde başka bir işletim sistemi çalıştırır. Saldırgan analizini gerçek cihaz yerine burada yapar, çünkü orada daha kolaydır. RASP bunu ortam ipuçlarından sezmeye çalışır.

Emülatör algılamada yanlış pozitif

suçsuz kullanıcıyı engellemek en pahalı hatadır

Zayıf sinyaller

pil yok · sensör yok
hipervizör biti

WSL2 / Hyper-V / VBS yüzünden
MEŞRU makinede de görülür

Doğru yaklaşım

birden çok sinyali TART
sertliği derecelendir

sunucu tarafı attestation
ikili “çalış/çalışma” değil

Bir skor üretin; eşiği ürünün risk iştahına göre ayarlayın.

Teknik	Nasıl	Not
CPUID hipervizör biti	<code>CPUID.1:ECX[31]</code> — "hypervisor present"	Bare-metal'de 0
Hipervizör satıcı imzası	<code>CPUID</code> yaprak <code>0x40000000</code> → 12 karakter (<code>KVMKVMKVM</code> , <code>VMwareVMware</code> , <code>Microsoft Hv...</code>)	Bazı hipervizörler gizler
Zamanlama (timing)	Bir işlemi ölç; tek adımlı hata ayıklama/ağır emülasyon çok yavaşlatır	Eşik makineye göre değişir

Demo 3 — VM/emülatör ve zamanlama algılama (CPUID)

i Demo 3 · `code/week-06/03-ortam-zamanlama` · **yalnız CPU komutu ve saat okur**

sh demo.sh — gerçek çıktı (WSL2)

```
(A) CPUID.1:ECX[31] hypervisor biti : VAR
    Hypervisor satıcı imzası       : (satıcı imzası gizli/bos)
(B) 2.000.000 işlem süresi       : 0.806 ms
SONUC: Hypervisor GORULDU. Ama WSL2/Hyper-V/VBS de böyle görünür;
bu TEK basına 'analiz ortamı' demek değildir - zayıf sinyal.
```

⚡ En önemli ders: yanlış pozitif

Yukarıdaki çıktı WSL2 üzerinde alındı ve **hipervizör "VAR"** dedi. Ama bu bir analiz ortamı **değil** — sıradan bir geliştirici makinesi! Modern Windows'ta **Hyper-V, WSL2 ve sanallaştırma tabanlı güvenlik (VBS)** yüzünden çoğu makine hipervizör bildirir. Yani bu bit tek başına "saldırgan analiz ediyor" anlamına **gelmez**. Bir uygulama sırf hipervizör gördüğü için çalışmayı reddetseydi, milyonlarca **meşru** kullanıcıyı engellerdi. RASP daima **birden çok göstergeli tartar** ve her birinin **atlatılabileceğini** kabul eder.

✓ Kural

Ortam algılamayı bir **risk skoru** olarak kullanın, ikili bir "çalış/çalışma" kararı olarak değil. Birden çok göstergeli birleştirin; yanlış pozitif (meşru VM/emülatör kullanan gerçek kullanıcıları) hesaba katın; nihai kararı **sunucu tarafı risk motoruyla** birlikte verin.

🔧 Sahada nasıl uygulanır?

Mobil dünyada karşılığı **emülatör algılamadır** (QEMU/Android emülatörü ipuçları: `ro.kernel.qemu`, sahte sensörler, tipik IMEI/seri değerleri). Yine aynı ilke: sinyal, kanıt değil; kombinasyon ve sunucu tarafı denetim.

5.1 İşlenmiş örnek: risk skoru ve eşik nasıl kurulur?

Bölüm 1.2'nin "tek kontrole güvenme" ilkesini burada sayılarla somutlaştıralım. Ortam algılamanın her sinyaline bir **ağırlık** (0–100 arası bir katkı puanı) verilir; bütün ağırlıklar toplanır ve bir **risk skoru** çıkar. Karar, tek bir `if (hipervizor_var)` değil, bu skorun bir **eşiğin** üstünde olup olmadığıdır.

Sinyal	Kaynak	Tek başına ne kadar güvenilir?	Örnek ağırlık
Hipervizör biti (CPUID)	Demo 3	Düşük — WSL2/Hyper-V/VBS'te de set olur	15
Bilinen hipervizör satıcı imzası	Demo 3	Orta — sanallaştırılmış ortamı daha kesin gösterir	25
Zamanlama anomalisi	Demo 3	Orta — yük altındaki gerçek makinede de olabilir	20
Bilinen emülatör dosya izi (mobilde)	<code>ro.kernel.qemu</code> benzeri	Yüksek — gerçek cihazda neredeyse hiç olmaz	40
Hata ayıklayıcı bağlı	Demo 2 (TracerPid)	Yüksek — meşru kullanıcı çalışırken nadiren olur	35

Örnek 1 — WSL2'de çalışan bir geliştirici (meşru kullanıcı). Demo 3'ün gerçek çıktısında gördüğümüz gibi hipervizör biti **VAR** ama satıcı imzası **gizli/boş**, zamanlama normal, emülatör izi yok, hata ayıklayıcı bağlı değil. Toplam skor: yalnız **15** (yalnız hipervizör biti). Skor düşük; kullanıcı engellenmez.

Örnek 2 — analiz ortamında çalışan bir saldırgan. Hipervizör biti VAR (15) + satıcı imzası görünür (25) + zamanlama anomalisi (20) + hata ayıklayıcı bağlı (35) = toplam **95**. Skor yüksek; işlem reddedilir ya da decoy döner (Bölüm 10).

Aradaki fark **tek bir bitin varlığı değil, kaç bağımsız sinyalin birlikte** görüldüğüdür. Eşik nereye konulmalı? Bu, üründen ürüne değişen bir **risk iştahı (risk appetite)** kararıdır — "kaç meşru kullanıcıyı engellemeye razıyım, karşılığında kaç saldırıyı önlerim" dengesi:

Ürün	Yanlış pozitifin maliyeti	Yanlış negatifin maliyeti	Örnek eşik
Bankacılık / ödeme uygulaması	Kullanıcı işlemini tamamlayamaz, çağrı merkezine yük biner	Sahte işlem, doğrudan para kaybı	Düşük (ör. 30) — şüphede reddet
Sadakat puanı / kupon uygulaması	Kullanıcı kızar ama kayıp küçük	Bir kupon fazladan kullanılır	Yüksek (ör. 70) — nadiren reddet
DRM korumalı içerik oynatıcı	Meşru kullanıcı içeriği izleyemez, mağaza puanı düşer	İçerik kopyalanır	Orta (ör. 50)

**Yanlış pozitifin gerçek maliyeti**

Eşiği çok düşük tutmak (ör. bankacılık eşiğini 70 yerine 10 yapmak), her WSL2/Hyper-V kullanan geliştiriciyi, her sanal makinede test eden BT departmanını ve bazı erişilebilirlik/güvenlik araçlarını kullanan sıradan kullanıcıyı **meşru olduğu hâlde** engeller. Bunun bedeli soyut değildir: çağrı merkezi yükü, mağaza puanı/olumsuz yorumlar, kullanıcı kaybı. RASP tasarımında "kaç saldırgan yakaladık" kadar "kaç meşru kullanıcıyı engelledik" de ölçülmelidir — mümkünse aşamalı devreye alma (önce yalnız kaydet, sonra reddet) ile.

**Kural**

Ortam/root/debugger sinyallerini **tek bir ikili karar** değil, **ağırlıklı bir risk skoru** olarak toplayın. Eşiği varlığın değerine ve ürünün risk iştahına göre **bilinçli** seçin; sabit, herkese uyan bir "endüstri standardı eşik" yoktur. Skoru ve eşiği zamanla (sunucudaki telemetriye bakarak) **ayarlanabilir** tutun; ikili dosyaya sabit kodlamayın.

6. Kanca ve enstrümantasyon algılama (LD_PRELOAD, Frida)

Hata ayıklayıcıdan daha sinsi bir tehdit: **fonksiyonlara kanca takma (hooking)** ve **dinamik enstrümantasyon**. Kanca (hook), bir fonksiyonun çağrısını araya girip değiştirmektir; enstrümantasyon ise çalışan bir programa kod enjekte edip davranışını izleme ya da değiştirme tekniğidir (bunun en yaygın aracı **Frida**'dır). Saldırgan programı durdurmadan, çağırdığı fonksiyonları (kendi anti-debug/anti-root kontrollerimiz dâhil) **kendi sürümüyle değiştirir**. Böylece Demo 2'deki kontrolleri "her zaman temiz" döndürecek şekilde yalancıya çıkarabilir.

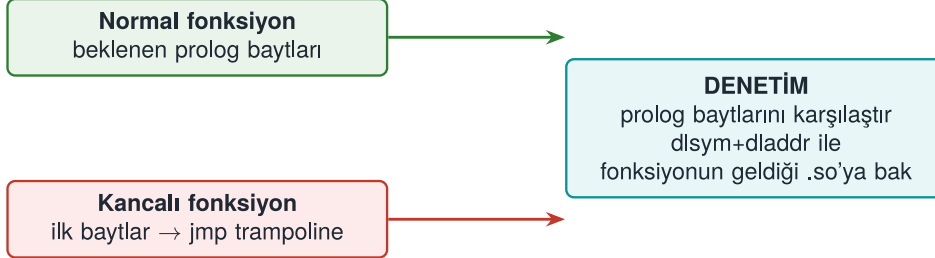
**Kısa tarihçe: statik yamadan dinamik enstrümantasyona**

- **1990'lar** — kanca teknikleri IAT (Import Address Table) ve PLT/GOT üzerinde çalışan **elle yazılmış**, hedefe özel araçlardı; her hedef için yeniden yazılması gerekiyordu.
- **2000'ler** — **LD_PRELOAD** (Linux) ve DLL enjeksiyonu (Windows), genel amaçlı ama **statik** (önceden derlenmiş) kanca takma yöntemleri olarak yaygınlaştı.
- **2009** — **Frida** projesinin temelleri atıldı; asıl fark, kanca kodunun **JavaScript ile çalışma zamanında** yazılıp enjekte edilebilmesiydi — derleme gerektirmiyordu.
- **2010'lar–bugün** — Frida, mobil güvenlik testinin standart aracı hâline geldi; RASP'in "kanca algılama" bölümünün var olma nedeninin büyük kısmı budur.

Bu tarihçenin dersi: kanca teknikleri **statikten dinamiğe** doğru evrildi; bu yüzden algılamanın da (Bölüm 6.1'deki `dladdr` tekniği gibi) belirli bir aracın imzasına değil, **davranışın kendisine** (fonksiyon nereden çalışıyor?) bakması gerekir.

Kanca (hook) nasıl algılanır?

kanca, fonksiyonun ilk baytlarını değiştirir



libc/vDSO dışında bir modülden geliyorsa kanca vardır (LD_PRELOAD).

Teknik	Nasıl	Platform
LD_PRELOAD	Bir <code>.so</code> 'yu önyükleyip standart fonksiyonları değiştirme (interposition)	Linux
PLT/GOT kancası	Çağrı tablosu girişlerini kendi fonksiyonuna yönlendirme	Linux/ELF
Satır içi (inline) kanca	Fonksiyonun ilk baytlarını <code>jmp</code> ile ezme	Her yer
Frida / Xposed	Dinamik ikili enstrümantasyon çerçeveleri	Mobil/masaüstü

Algılama fikirleri (Katalog K7/K8/K18):

- `LD_PRELOAD` ortam değişkeni boş mu? (`getenv` — **zayıf**, çünkü `getenv` 'in kendisi de kancalanabilir)
- Bir fonksiyon **hangi paylaşımlı nesneden** çözülüyor? `dlsym(RTLD_DEFAULT, ...)` + `dladdr` ile fonksiyonun geldiği `.so` dosyasına bakılır; beklenen libc yerine yabancı bir `.so` çıkıyorsa **kanca vardır**. (Bu, `getenv` sahteciliğine dayanmaz.)
- `/proc/self/maps` 'te beklenmeyen yürütülebilir bölgeler, bilinen çerçeve göstergeleri.

Demo 4 — LD_PRELOAD ile kanca ve `dladdr` ile algılama

Demo 4 · [code/week-06/04-preload-kanca](#) · **yalnız Linux/WSL** · **CWE benzeri: dinamik enstrümantasyon**

Demo, `time()` fonksiyonunu ele geçiren küçük bir sahte kanca kütüphanesi (`libsahtekanca.so`) **derler**. `LD_PRELOAD` ile bu kütüphane **yalnız tek bir demo komutuna** yüklenir. Ana program, fonksiyonun geldiği `.so` 'ya bakarak kancayı yakalar. Windows'ta `demo.ps1` bunun WSL'de nasıl çalıştırılacağını anlatır (Windows karşılığı IAT/satır-içi kanca tespitidir).

kanca_ana.c (özet)

```
void *p = dlsym(RTLD_DEFAULT, "time"); /* onyuklu kanca varsa onu doner */
Dl_info info;
dladdr(p, &info); /* fonksiyonu saglayan .so */
int kanca = !mesru_mi(info.dli_fname); /* libc/vdso/ld disi = KANCA */
```

sh demo.sh — gerçek çıktı (WSL)

```
ADIM 1 - Normal calisma (LD_PRELOAD yok): temiz beklenir
time      -> linux-vdso.so.1
getenv    -> /lib/x86_64-linux-gnu/libc.so.6
time(NULL) dondurdu : 1789852509
SONUC: Temiz - preload/kanca görünmüyor.

ADIM 2 - Saldiri: sahte kanca LD_PRELOAD ile YALNIZ bu surece yukleniyor
time      -> bin/linux/libsahtekanca.so (KANCA!)
time(NULL) dondurdu : 1234567890
SONUC: Fonksiyon kancasi / preload ALGILANDI (2 sinyal).
```

Temiz koşuda `time`, çekirdeğin **VDSO**'sundan çözüldü (bu meşrudur, kanca değil). Kanca yüklenince aynı `time` `libsahtekanca.so` 'dan çözüldü ve **sabit sahte değer** (1234567890) döndürdü; `dladdr` fonksiyonun yerini göstererek kancayı yakaladı. Dikkat: temiz koşuda `linux-vdso` 'yu **yanlış pozitif** saymamak için algılama `libc`, `VDSO` ve dinamik yükleyiciyi meşru kaynaklar olarak kabul eder — bu, "algılama yazmak, algılamayı doğru yazmaktan kolaydır" dersidir.

6.1 İşlenmiş örnek: `dlsym` + `dladdr` ile fonksiyonun kaynağını adım adım bulmak

Demo 4'ün çekirdeği iki fonksiyon çağırısıdır: `dlsym` ve `dladdr`. İkisinin birlikte ne yaptığını adım adım açalım.

Adım 1 — `dlsym(RTLD_DEFAULT, "time")` ne yapar? `dlsym`, çalışan sürecin **sembol tablosunda** aranan isimde (`"time"`) bir sembol arar ve onun **bellek adresini** döndürür. `RTLD_DEFAULT`, "programın normal sembol arama sırasını (kendi ikili, sonra yüklenen kütüphaneler, `LD_PRELOAD` 'lı kütüphaneler dâhil) kullan" anlamına gelen özel bir tanıtıcıdır. Bu adımın çıktısı yalnız bir **sayı** (bir bellek adresi); hangi dosyadan geldiğine dair bir bilgi taşımaz.

Adım 2 — bu adresi kim sağladı? İşte `dladdr` 'nin işi budur: verilen bir adresi alır, çalışan sürecin belleğe yüklenmiş **paylaşımlı nesnelerinin** (`.so` dosyalarının) haritasında arar ve adresin **hangi** `.so` 'nun bellek aralığına düştüğünü bulur. Sonucu bir `Dl_info` yapısına yazar; bizim kullandığımız alan `dli_fname` — o `.so` 'nun **dosya yolu**.

Adım 3 — temiz koşuda. `LD_PRELOAD` boşken, `time` sembolü normal arama sırasıyla çözülür ve çekirdeğin sağladığı **VDSO** (virtual Dynamic Shared Object — çekirdeğin süreç belleğine kendiliğinden eklediği, sistem çağırısı yapmadan hızlı okuma sağlayan özel bir sanal `.so`) içinden gelir. Demo çıktısı: `time -> linux-vdso.so.1`. `mesru_mi()` fonksiyonu bu ismi tanır (`linux-vdso` alt dizesini arar) ve **kanca değil** der.

Adım 4 — `LD_PRELOAD` ile kanca yüklenince. `demo.sh`, `LD_PRELOAD=./libsahtekanca.so` ortam değişkeniyle programı başlatır. Dinamik bağlayıcı (`ld-linux.so`), normal arama sırasının **en başına** bu kütüphaneyi sokar; kütüphane kendi `time()` fonksiyonunu tanımladığı için, `dlsym(RTLD_DEFAULT, "time")` artık **VDSO'daki gerçek `time` 'ı değil**, **libsahtekanca.so'daki sahte `time` 'ı** bulur. `dladdr` bu adresin `libsahtekanca.so` 'nun bellek aralığına düştüğünü görür; `dli_fname = "bin/linux/libsahtekanca.so"` döner. `mesru_mi()` bu ismi tanımadığı (`libc/VDSO/ld` değil) için **KANCA** der — tam demo çıktısındaki `time -> bin/linux/libsahtekanca.so (KANCA!)` satırı.

Adım 5 — bu, neden `getenv(LD_PRELOAD)` denetiminden daha güçlü? `getenv` denetimi ortam değişkeninin **kendisine** bakar — ama ortam değişkeni yalnız programın **başlatılma anında** kütüphaneyi yüklemek için kullanılır; bir kez yüklendikten sonra saldırgan onu **silebilir** (`unsetenv`) ya da kendi sahte kütüphanesinde `getenv` 'i de kancalayıp boş döndürebilir (tam olarak Etkinlik 1'in denediği şey). `dladdr` denetimi ise ortam değişkenine hiç bakmaz; doğrudan "bu fonksiyon şu an **fiilen nereden** çalışıyor?" sorusunu sorar — kanca zaten yüklenmiş olduğu için bu sorunun cevabı, ortam değişkeninden değil çalışan **kodun kendisinden** gelir ve bu yüzden `getenv` sahteciliğine dayanmaz.

✓ Kural

Kanca algılamada `getenv(LD_PRELOAD)` gibi **manipüle edilebilir** kaynaklara güvenmeyin; kritik fonksiyonların **kaynağını** (hangi modülden geldiğini) doğrulayın. Birden çok fonksiyonu kontrol edin; `/proc/self/maps` gibi ek sinyaller ekleyin; sonucu bir tepki politikasına bağlayın.

🔧 Değerlendirici nasıl test eder?

Değerlendirici bilinen bir enstrümantasyon aracıyla (ör. Frida) kritik fonksiyonları kancalar ve korumanın bunu yakalayıp yakalamadığına bakar; ardından **algılama fonksiyonunun kendisini** kancalayarak korumayı kör edip edemeyeceğini dener (tek nokta mı, dağıtık mı?).

7. Dinamik bellek koruması ve bellek izleme tespiti

İzlencenin bu haftaya koyduğu maddelerden biri **dinamik bellek korumasıdır**: program çalışırken bellekteki hassas verinin okunmasına ve değiştirilmesine karşı alınan önlemler. **1. haftada** sırları bellekten silmeyi ve takasa düşmesini önlemeyi, **3. haftada** "kullanımda veri" katmanlarını gördük. Bu bölüm aynı soruyu çalışma anı saldırganı açısından yeniden sorar: saldırgan belleği **izliyor** ya da **değiştiriyorsa** ne yapabiliriz?

Dinamik bellek koruması: hangi katman neyi kapatır?

bellek saldırganının en kolay ulaştığı yerdir

Döküm kapatma

`PR_SET_DUMPABLE=0` — çökme dökümünde sır kalmaz

Bellek kilidi

`mlock` — sır takas alanına yazılmaz

Kısa ömür + silme

`explicit_bzero` — pencere daralır

Bütünlük sayacı

korunan değer yanında sağlama tutulur; değişirse yakalanır

İzleme tespiti

`ptrace` / okuma denemesi algılanır ve puana katılır

Hiçbiri tek başına yetmez; her biri saldırganın bir yolunu pahalılaştırır.

Saldırgan bellekle ne yapar?

Saldırı	Örnek	Hedef
Okuma	Süreç belleğinin dökümünü almak, bellekte bilinen bir değeri aramak	Anahtar, PIN, çözülmüş veri
Değiştirme	Bir oyundaki puanı, bir sayaç değerini ya da bir "lisans geçerli" bayrağını bellekte değiştirmek	Mantık ve denetimler
İzleme	Bir değer değiştiğinde haber veren araçlarla hangi adresin neyi tuttuğunu bulmak	Değişkenlerin yerini keşfetmek

Bu saldırıların hepsi için saldırganın süreç belleğine **erişmesi** gerekir: bir hata ayıklayıcıyla, işletim sisteminin süreç belleği arayüzleriyle ya da sürecin içine yüklenmiş bir kütüphaneyle. Bu yüzden bellek koruması, önceki bölümlerdeki hata ayıklayıcı ve kanca algılamayla birlikte düşünülür.

Önlem 1: Bellekte az, kısa ve dağınık tut

En etkili bellek koruması, bellekte **okunacak bir şey bırakmamaktır**:

- Kısa ömür:** Sır, yalnız kullanıldığı fonksiyonun içinde açık olur ve fonksiyon dönmeden silinir (3. hafta: açıklık penceresi).
- Örtme ve parçalama:** Kullanılmadığı sürede sır, rastgele bir maskeyle XOR'lanmış ya da parçalara bölünmüş olarak durur; bellekte tek bir yerde sırrın kendisi bulunmaz.
- Rastgele değerle silme:** İş bitince bellek sıfırla değil rastgele değerlerle doldurulur; "burada az önce bir sır vardı" izini de bırakmaz. Sahada native tarafta rastgele üreticinin tek kullanım yeri budur.
- Hiç açılmamak:** Whitebox kriptografide anahtar hiçbir anda açık halde bellekte bulunmaz (**11. hafta**). Sahadaki kabuk matrisinde ödeme sırasında native tarafta kalan tek kabuk budur (3. hafta).

Önlem 2: Belleğin okunmasını işletim sistemiyle zorlaştır

Önlem	Platform	Etkisi
Döküm kapatma, <code>PR_SET_DUMPABLE = 0</code>	Linux	Aynı kullanıcıya ait başka bir süreç bile hata ayıklayıcı olarak bağlanamaz ve süreç belleği dosyasını okuyamaz (1. hafta)
Bellek kilitleme (<code>mlock</code> / <code>VirtualLock</code>), <code>MADV_DONTDUMP</code>	Linux / Windows	Sır takasa ve döküme düşmez (1. hafta)
Sayfa korumaları (<code>mprotect</code> / <code>VirtualProtect</code>)	Linux / Windows	İklendirmeden sonra değişmemesi gereken tablolar salt okunur yapılır; değiştirme girişimi hata üretir
Korunan süreç özellikleri	Windows, mobil platformlar	İşletim sisteminin sağladığı "korunan süreç" ve uygulama kum havuzu, başka uygulamaların belleğe erişimini sınırlar

Bu önlemler yönetici (root) yetkisine sahip bir saldırganı durdurmaz; ama saldırganı **daha güçlü** bir konuma çıkmaya zorlar ve bu da root algılamasının devreye girdiği yerdir.

Somut bir örnek: PR_SET_DUMPABLE neyi değiştirir? Linux'ta her süreç varsayılan olarak "dump edilebilir"dir — çekirdek çökme anında bir çekirdek dökümü (core dump) üretebilir **ve** aynı kullanıcıya ait başka bir süreç `ptrace(PTRACE_ATTACH, pid, ...)` ile ona bağlanabilir. `prctl(PR_SET_DUMPABLE, 0)` çağırısı bu bayrağı kapatır:

Durum	<code>ptrace</code> ile dışarıdan bağlanma	Çökme anında bellek dökümü	<code>/proc/<pid>/mem</code> okuma
<code>PR_SET_DUMPABLE = 1</code> (varsayılan)	Aynı kullanıcının başka bir süreci bağlanabilir	Üretilir (diskte kalır)	Okunabilir
<code>PR_SET_DUMPABLE = 0</code>	Aynı kullanıcının süreci bile bağlanamaz ("Operation not permitted")	Üretilmez	Reddedilir

Bu, Bölüm 4'teki `TracerPid` denetimiyle **tamamlayıcı** çalışır: `TracerPid` "şu an biri izliyor mu?" diye **algılar**; `PR_SET_DUMPABLE = 0` ise "aynı kullanıcıya ait sıradan bir araç izleyemesin" diye **önler**. Kök yetkisine sahip bir saldırgan bu bayrağı yine de aşabilir (`ptrace_scope` ayarına ve yetkisine bağlı) — bu yüzden Bölüm 8'deki kök göstergesiyle birlikte değerlendirilmelidir.

Önlem 3: Değiştirilen veriyi fark et

Bellekteki kritik bir yapının (bir sayaç, bir yetki bayrağı, bir yapılandırma tablosu) sessizce değiştirilmesini fark etmenin yolu, o veriyi **tek başına** bırakmamaktır:

Kritik bir değeri korumalı tutmak: değer + gölge kopya + özet

```
typedef struct {
    uint32_t deger;           /* asıl değer */
    uint32_t golge;          /* deger XOR çalışma anı maskesi */
    uint32_t ozet;           /* deger ve golge üzerinden kısa bir MAC */
} KorunanSayac;

/* Okurken üçünün tutarlılığı denetlenir; tutmuyorsa değer bellekte değiştirilmiştir. */
int sayac_oku(const KorunanSayac *s, uint32_t *cikti)
{
    if ((s->golge ^ calisma_maskesi) != s->deger) return -1;
    if (kisa_mac(s->deger, s->golge) != s->ozet) return -1;
    *cikti = s->deger;
    return 0;
}
```

Saldırgan bellekte yalnız `deger` alanını değiştirirse gölge kopya ve özet tutmaz. Üçünü birden tutarlı biçimde değiştirmek için maskeyi ve özet anahtarını da bulması gerekir; bu da işin maliyetini artırır. Aynı fikir, [5. haftadaki](#) cihaz parmak izinin **iki farklı özetle iki farklı yerde** saklanması ve [2. haftadaki](#) kurcalamaya dayanlı günlükte de vardır.

7.1 İşlenmiş örnek: `KorunanSayac` bir saldırıyı nasıl yakalar?

Yukarıdaki yapıyı sayılarla izleyelim. `calisma_maskesi`, çalışma başında rastgele üretilmiş sabit bir değer olsun: `0xA5A5A5A5`. Bir ödeme denemesi sayacı `deger = 3` (üçüncü deneme) olduğunda yapı şöyle doldurulur:

Alan	Değer	Nasıl hesaplandı
deger	0x00000003	asıl sayaç
golge	0xA5A5A5A6	deger XOR calisma_maskesi = 0x00000003 XOR 0xA5A5A5A5
ozet	(örnek) 0x7F2C1B90	kisa_mac(deger, golge)

sayac_oku() her okumada golge XOR calisma_maskesi 'nin deger 'e eşit olup olmadığına bakar: 0xA5A5A5A6 XOR 0xA5A5A5A5 = 0x00000003 — deger ile eşit, tutarlı; fonksiyon 0 döner.

Şimdi saldırganın bir bellek düzenleyicisiyle yalnız deger alanını 0x00000003 (99) yaptığını düşünelim (bir deneme sayacını ya da bir puanı sıfırlamaya/artırmaya çalışıyor). golge ve ozet alanları **değişmedi**. Bir sonraki sayac_oku() çağırısı: golge XOR calisma_maskesi = 0xA5A5A5A6 XOR 0xA5A5A5A5 = 0x00000003 — ama deger artık 0x00000003. İkisi **eşleşmiyor**; fonksiyon -1 döner, tutarsızlık yakalanır. Saldırganın bunu atlatabilmesi için golge 'yi de doğru maskeyle yeniden hesaplaması **ve** ozet 'i de tutarlı üretmesi gerekir — ki kısa_mac 'in anahtarını bilmediği sürece bu, Bölüm 3.1'deki HMAC altın değerini yeniden üretmekle aynı zorluktur.

⚠ Tutarlılık denetimi de bir denetimdir

sayac_oku 'nun -1 dönüşünü "hiçbir şey olmamış gibi" yok sayan bir çağırın, bütün korumayı boşa çıkarır; ayrıca denetimin kendisi de yamalanabilir. Bu yüzden tutarsızlık bir tepkiye bağlanır (tepki politikası) ve denetim kodu da bütünlük denetiminin kapsamına alınır.

Önlem 4: İzlendiğini fark et

Bellek izleme araçlarının çoğu, sürece bir hata ayıklayıcı olarak bağlanarak ya da sürecin içine bir kütüphane yükleyerek çalışır. Bu yüzden izleme tespitinin temeli, hata ayıklayıcı ve kanca algılama bölümlerinde gördüğümüz yöntemlerdir. Bunlara ek olarak bazı dolaylı sinyaller kullanılır:

- **Zamanlama:** Tek adımlı çalıştırılan ya da kesme noktalarıyla durdurulan kod, normalden çok daha yavaş çalışır; iki nokta arasındaki beklenmedik uzun süre bir sinyaldir (ortam algılamadaki zamanlama ölçümüyle aynı fikir).
- **Beklenmeyen modüller:** Süreç belleğinde, uygulamanın kendisinin yüklediği kütüphaneler bulunması.
- **Tutarlılık bozulması:** Önlem 3'teki gibi korunan değerlerin tutarsızlaşması.

Bu sinyallerin hiçbirisi tek başına kesin değildir; hepsi tepki politikasındaki **risk skoruna** katkı olarak kullanılır.

? Değerlendirici nasıl test eder?

Hassas işlem sırasında ve sonrasında süreç belleğinin dökümünü alıp bilinen test sırlarını arar; bellekteki bir sayacı ya da bayrağı bir bellek düzenleyiciyle değiştirip uygulamanın fark edip etmediğine bakar; bellek izleme araçlarını bağlayıp uygulamanın tepkisini gözlemler. Raporda "sır bellekte ne kadar süre açık kaldı" ve "değiştirilen değer fark edildi mi" soruları cevaplanır.

8. Kök/ayrıcalklı ortam ve bileşen imza doğrulaması

Kök/ayrıcalklı ortam: göstergeden karara

tek göstergeyle engellemek meşru kullanıcıyı vurur



Root'lu cihaz suçlu değildir; yalnızca güvence düzeyi düşüktür. Tepkiyi ona göre ölçekleyin.

8.1 Kök (root) / ayrıcalklı ortam göstergesi

Kök (root), bir cihaz üzerinde normalde kısıtlı olan tam yetkidir; **jailbreak** aynı yetkiyi iOS gibi kapalı sistemlerde zorla açmanın adıdır. Root'lu (ya da jailbreak'li) bir cihazda uygulamanın güvenlik varsayımları çöker: her süreç belleği okuyabilir, dosya sistemi korumaları aşılar. RASP root göstergelerine bakar (Katalog K4/K5): `su` ikilisinin varlığı, tehlikeli paketler, yazılabilir sistem yolları, root gizleme çerçeveleri. Masaüstündeki taşınabilir karşılığı: uygulama **yükseltilmiş yetkiyle** mi çalışıyor (Linux `geteuid()==0`, Windows yükseltilmiş token)?

Mobil "root" ile masaüstü "yükseltilmiş yetki" arasındaki fark yalnız isimdir, fikir aynıdır. Android'de bir uygulamanın `su` ikilisini bulması, root erişiminin **kurulu olduğunu** gösterir; Linux/masaüstünde bir sürecin `geteuid()`'i `0` döndürmesi, sürecin **fiilen** root kullanıcısı olarak çalıştığını gösterir. İkisi arasındaki küçük ama önemli fark: mobilde root **kurulu olabilir ama o an kullanılmıyor olabilir** (gösterge, göstergedir); masaüstünde `geteuid()==0` ise **o anki gerçek durumu** bildirir (çekirdeğin kendisi bu bilgiyi tutar, tıpkı Bölüm 4.1'deki `TracerPid` gibi sahtelemesi zordur). Bu yüzden masaüstü/sunucu tarafındaki "ayrıcalklı süreç" denetimi, mobildeki "root göstergesi" taramasından biraz daha **güvenilir** bir sinyaldir — yine de tek başına karar vermek yerine Bölüm 8.1.1'deki risk skoruna eklenmelidir.

Demo 7 — Root/ayrıcalklı ortam göstergesi (yalnız okuma)

Demo 7 · `code/week-06/07-ortam-yetki` · **root/ayrıcalk göstergesi, salt okuma**

Program (1) ayrıcalk seviyesini (root/admin mi) ve (2) bilinen "tehlikeli işaret" yollarının varlığını yalnız **okuyarak** kontrol eder. Demoyu deterministik kılmak için komut satırından sahte bir işaret (`cikti/sahte_su`) verilir.

sh demo.sh — gerçek çıktı (kısaltılmış)

```

ADIM 2 - Isaretli durum: sahte bir 'su' isaret dosyasi olusturuluyor
1) Ayricalik seviyesi : normal kullanıcı
2) Tehlikeli gosterge taramasi:
  [BULUNDU] cikti/sahte_su (ek isaret)
SONUC: Ayricalikli/riskli ortam GOSTERGESI var (1 sinyal).
```

⚠ Sinyal, kanıt değil

Root göstergeleri **kolayca sahtelenir** (Magisk Hide gibi araçlar `su` 'yu ve bilinen yolları gizler). Ayrıca bir uygulama sırf root gördüğü için çalışmayı reddederse, cihazını **meşru** biçimde root'lamış kullanıcıları da engeller (yanlış pozitif / kullanıcı deneyimi dengesi). Modern yaklaşım: cihazın bütünlüğünü **işletim sistemi/platform attestation** servisiyle (nonce'lu, sunucu tarafı doğrulanan) ölçmek — istemcinin kendi "root değilim" iddiasına güvenmemek.

8.1.1 Kök göstergesini de bir risk skoruna bağlamak

Bölüm 5.1'deki risk skoru fikri kök/ayrıcılık göstergesi için de geçerlidir; hatta sahada tek bir gösterge yerine bir **liste** önerilir, çünkü her gösterge tek başına hem **kaçırılabilir** (saldırgan gizler) hem **yanlış pozitif üretebilir** (meşru kullanıcı cihazını gerçekten root'lamıştır).

Gösterge	Tek başına anlamı	Örnek ağırlık
<code>su</code> ikilisi/işareti bulundu	Root erişimi kurulu olabilir	20
Bilinen root yönetim paketi yüklü	Root aktif olarak yönetiliyor	25
Sistem bölümü yazılabilir	Salt-okunur olması gereken alan değiştirilebilir	30
<code>geteuid() == 0</code> (masaüstü/sunucu karşılığı)	Süreç fiilen yükseltilmiş yetkiyle çalışıyor	40

Demo 7'nin gerçek çıktısındaki tek sinyal (`cikti/sahte_su` bulundu → 1 sinyal) düşük bir skora karşılık gelir; tek başına bir bankacılık uygulamasını durdurmaya **yetmemelidir**. Skorun yüksek çıkması için birden çok göstergenin **birlikte** görünmesi beklenir — tıpkı Bölüm 5.1'deki ortam örneğinde olduğu gibi.

⚠ Sık hata: root görünce anında `exit()`

Kök göstergesi tek başına görülür görülmez uygulamayı kapatmak, hem gerçek saldırıyı yakalamaz (ipuçlarını zaten gizlemiştir, bu denetim onu hiç görmez) hem de cihazını meşru amaçla (özelleştirme, geliştirme) root'lamış sıradan bir kullanıcıyı cezalandırır. Kural: göstergelyi skora ekle, kararı **eşiğe** bırak (Bölüm 5.1); fail-closed yerine önce **azaltılmış işlevsellik** (ör. yüksek tutarlı işlemleri reddet, düşük tutarlıları kabul et) düşün.

8.2 Bileşen/paket imza ve özet doğrulaması (APK imza doğrulamasının genel hâli)

Android'de uygulama, **yeniden paketlemeye (repackaging)** karşı **APK imzasını** doğrular (APK Signature Scheme v1/v2/v3): saldırgan uygulamayı açıp değiştirse ve yeniden paketlese bile, orijinal geliştiricinin özel anahtarıyla üretilmiş imzayı taşıyamaz. Bu, "yüklediğim/çağırdığım bileşen beklenen, değiştirilmemiş sürüm mü?" sorusunun genel hâlidir (Katalog K2/K3/K6).

Demo 6 — Bileşen imza doğrulaması ve repackaging tespiti

Demo 6 · `code/week-06/06-bilesen-imza` · imza/özet doğrulama

Bir "eklenti modülü" dosyası yüklenmeden **önce** ayırık bir imza (HMAC-SHA-256) ile doğrulanır. Modül bir bayt bile değişirse imza tutmaz ve yükleme reddedilir.

sh demo.sh — gerçek çıktı

```
ADIM 3 - Yükleme oncesi dogrulama (modul degismedi): TUTAR
Modul imzasi TUTTU -> guvenle yuklenebilir.
ADIM 4 - Saldiri: modul YENIDEN PAKETLENIR (tek bayt eklenir)
Modul imzasi TUTMADI -> YENIDEN PAKETLENMIS/degistirilmis.
Yükleme REDDEDILDI. (cikis kodu 3)
```

Önemli fark: HMAC vs. asimetrik imza

APK v2/v3 **açık anahtar (asimetrik)** imza kullanır (bkz. [Hafta 3'teki](#) dijital imza): doğrulayanın imzalama anahtarına değil, yalnız **açık anahtara** ihtiyacı olur. Bu demoda öğretim için **paylaşılan anahtarlı HMAC** kullanıyoruz (basit ve `cen429_kripto.h`'da hazır); asimetrik imza (Ed25519 / RSA-PSS) ve sertifika zincirlerini [Hafta 10'da](#) işleyeceğiz. İlke aynı: yüklemekten önce **bütünlük + kaynak** doğrulaması.

8.2.1 İşlenmiş örnek: repackaging bir bayt eklemekle nasıl yakalanır?

Demo 6'nın gerçek çıktısındaki "modül YENIDEN PAKETLENİR (tek bayt eklenir)" adımını Bölüm 3.1'deki mantıkla birleştirelim. Modülün doğrulaması da tıpkı Demo 1 gibi **dosyanın tamamı** üzerinden hesaplanan bir HMAC-SHA-256'dır; fark, hedefin bu kez programın kendisi değil, **ayrı bir dosya** (eklenti modülü) olmasıdır.

Adım 1 — orijinal modül. Modül dosyası N bayttır; yükleyici onu okur, `HMAC(anahtar, modul[0..N))` hesaplar ve önceden kaydedilmiş imzayla karşılaştırır. Eşleşir → "TUTAR" → güvenle yüklenir.

Adım 2 — saldırgan modülü açar, bir özellik ekler, yeniden paketler. Bu işlem dosyanın **sonuna bir bayt ekler** ($N+1$ bayt oldu) — modülün amaçlanan işlevine hiç dokunmasa bile. Yükleyici bu kez `HMAC(anahtar, modul[0..N+1))` hesaplar. Girdinin **uzunluğu bile değiştiği** için (Bölüm 3.1'deki tek bayt **değeri** değişikliğinden daha köklü bir fark), HMAC'in dahili blok işleme mekanizması en baştan farklı bloklar görür ve sonuç **tümüyle** farklı çıkar — tıpkı Demo 1'de gördüğümüz gibi.

Adım 3 — karşılaştırma başarısız. Yeni HMAC, kaydedilmiş imzayla eşleşmez → "TUTMADI" → "YENIDEN PAKETLENMIS/degistirilmis" → yükleme reddedilir (çıkış kodu 3). Saldırgan tek bir bayt bile eklese, imzayı **yeniden üretebilmesi** için imzalama anahtarına ihtiyacı vardır; bu anahtar (öğretim amaçlı HMAC'te) yükleyicinin içindedir, ama gerçek APK imzasında (asimetrik) yalnız orijinal geliştiricide bulunan **özel anahtardır** — bu da Bölüm 8.2'nin "Önemli fark" notundaki ayrımın can alıcı noktasıdır: HMAC'i doğrulayan herkes aynı zamanda **üretebilir** de; asimetrik imzada doğrulayan yalnız **doğrular**, üretemez.

✓ Kural

Dinamik yüklenen her bileşeni (modül, eklenti, native kütüphane, güncelleme paketi) yüklemekten **önce** kriptografik olarak doğrulayın. Mümkünse **karşılıklı** doğrulama kurun (Katalog K6): native, üst katmanın özetini; üst katman, native'in özetini doğrular — saldırganın **ikisini birden** tutarlı yamalaması gerekir. Uzun vadede **asimetrik imza** tercih edin (dağıtım kolaylığı).

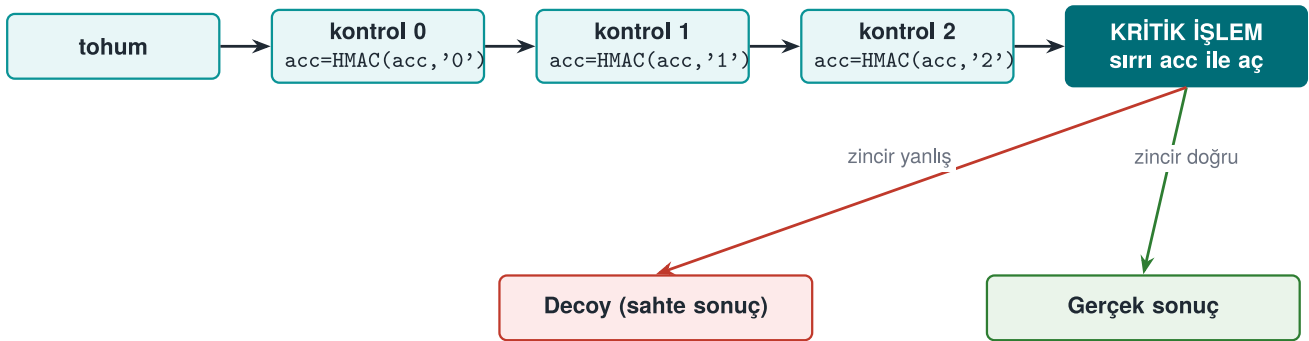
9. Kontrol akışı bütünlüğü: tek bir `if` neden yetmez

Şu ana kadarki bütün kontroller (bütünlük, anti-debug, ortam, root) bir noktada **bir karar** verir: `if (tehlike) reddet;`. Saldırganın işi basittir: o **tek atlamayı** (`je → jmp`) yamalıyıp kontrolü es geçmek (Bölüm 3'teki yama). O hâlde kontrolleri **atlanamaz** kılmanın yolu nedir?

Cevap: güvenlik kontrollerini bir **kontrol akışı sayacına / anahtar zincirine** bağlamak (Katalog K17). Kritik işlem, doğru sonucu **ancak** bütün kontrol noktalarından **sırayla** geçilmişse üretebilir. Bunu bir **veri bağımlılığıyla** kurarız: her kontrol noktası bir anahtar zincirini ilerletir (`acc = HMAC(acc, "asama-i")`); kritik işlem bu zincirden türetilen anahtarla bir sırrı açar.

Kontrol akışı sayacı: denetim atlandı mı?

her denetim akümülatörü ilerletir



Tek `if` bir `NOP`'la atlanır; sayaç, denetimin GERÇEKTEN çalıştığını sonradan kanıtlar.

Bir kontrol atlanır ya da sırası bozulursa **acc** farklı çıkar, anahtar yanlış olur ve GCM çözme **reddedilir** — gerçek sonuç üretilemez. Böylece tek bir `jmp` yaması işe yaramaz.

Demo 5 — Kontrol akışı sayacı ve skip saldırısı



Demo 5 · `code/week-06/05-akis-sayaci` · control-flow integrity (Katalog K17)

`sh demo.sh` — gerçek çıktı (kısaltılmış)

```

SENARYO 1 - Normal: butun kontrol noktaları sirayla çalışır
[geçildi] kontrol noktası 0, 1, 2
SONUC: ODEME ONAYLANDI -> "ODEME-ONAYI-TOKEN-4242"

SENARYO 2 - Saldırı: kontroller tamamen ATLANIR
[ATLANDI] hiçbir kontrol noktası çalışmadı
(uyarı: sayac=0 iz=0x0 beklenen=3/0x7)
SONUC: ODEME REDDEDİLDİ (zincir anahtarı yanlış). ... decoy doner

SENARYO 4 - Saldırı: kontroller YANLIS SIRADA çalışır
SONUC: ODEME REDDEDİLDİ (zincir anahtarı yanlış). ... decoy doner

```

Normal akışta üç kontrol sırayla geçildi, zincir doğru oldu ve ödeme onaylandı. Kontroller atlandığında, kısmen geçildiğinde ya da yanlış sırada geçildiğinde zincir farklı çıktı; kritik işlem gerçek sonucu **üretemedi** ve sahte (decoy) döndü. Program ayrıca bir **çift sayaç** (sayac + iz biti) tutar (K17); bu, anahtar zinciriyle birlikte üçlü bir tutarlılık kontrolü sağlar.

"Çift sayaç" neden gerekli, acc zinciri yetmiyor mu? acc zinciri **hangi aşamaların hangi sırayla** çalıştığını dolaylı olarak kanıtlar (yanlış aşama/sıra → yanlış anahtar), ama demo çıktısındaki uyarı satırında gördüğümüz gibi (uyarı: sayac=0 iz=0x0 beklenen=3/0x7) program ayrıca iki **basit** sayaç daha tutar: sayac (kaç kontrol noktasının çalıştığını sayan düz bir tam sayı) ve iz (her kontrol noktasının kendi bit konumunu 1 yaptığı bir bit kümesi, ör. 0., 1. ve 2. kontrol geçtiyse iz = 0b111 = 0x7). Bunların amacı acc'ten **farklıdır**: acc kriptografik olarak güçlüdür ama tek başına "kaç kontrol geçti, hangileri" gibi **okunabilir bir teşhis bilgisi** vermez (HMAC çıktısından geriye "üç kontrol de geçti" diye çıkarım yapılamaz). sayac ve iz, geliştiriciye ve günlüğe (log) okunabilir bir özet sağlarken, gerçek **yetkilendirme kararı** yine yalnız acc'ten türeyen anahtara dayanır — sayac ve iz alanlarının kendisi yamalanıp 3 ve 0x7 yapılsa bile, acc zinciri yanlış olduğu için anahtar yine yanlış çıkar. Üç değer (acc, sayac, iz) böylece birbirini **çapraz doğrular**: biri tutarsızsa günlükte fark edilir, hiçbiri tek başına yetkilendirme kararını değiştiremez.

9.1 İşlenmiş örnek: anahtar zincirini elle hesaplamak

acc = HMAC(acc, "asama-1") cümlesini soyut bırakmayalım; gerçek openssl komutlarıyla elle hesaplayalım. Başlangıç değeri acc0, 32 sıfır bayttan oluşan sabit bir anahtarla **boş** veriyi HMAC'leyerek üretilsin (gerçek sistemde bu, oturum başına rastgele üretilir):

Kontrol akisi zincirini elle izlemek (gerçek openssl çıktısı) - baslangic

```

A0=$(printf '' | openssl dgst -sha256 -mac HMAC \
    -macopt hexkey:0000000000000000000000000000000000000000000000000000000000000000 \
    -binary | xxd -p -c 256)
echo "acc0 = $A0"

```

```
acc0 = b613679a0814d9ec772f95d778c35fc5ff1697c493715653c6c712144292c5ad
```

Doğru sıra — kontrol noktası 0, sonra 1, sonra 2 sırayla geçilir. Her aşamada bir önceki acc anahtar, aşama etiketi veri olarak HMAC'lenir ve sonuç yeni acc olur:

Sirali gecis: acc1 -> acc2 -> acc3

```

A1=$(printf 'asama-0' | openssl dgst -sha256 -mac HMAC -macopt hexkey:$A0 -binary | xxd -p -c 256)
A2=$(printf 'asama-1' | openssl dgst -sha256 -mac HMAC -macopt hexkey:$A1 -binary | xxd -p -c 256)
A3=$(printf 'asama-2' | openssl dgst -sha256 -mac HMAC -macopt hexkey:$A2 -binary | xxd -p -c 256)
echo "acc3 (DOGRU zincir) = $A3"

```

```
acc3 (DOGRU zincir) = fc5c2a86e948b190dbe61619375a77bee2aa79911feec3d37f5d7d20615cada8
```

Kritik işlem (ör. ödeme onayı), yalnız `acc3` **tam olarak bu değere eşitse** doğru anahtarı üretir ve gerçek sonucu üretir — Demo 5'in SENARYO 1'i.

Saldırı — kontrol noktaları atlanır, yalnız birinci aşama çalışır. Saldırgan 0 ve 2 numaralı kontrol fonksiyonlarına giden çağrılar bir yamayla es geçer (Bölüm 3'teki `je → jmp` yamasıyla aynı fikir); yalnız `asama-1` etiketi işlenir:

Atlanmış gecis: yalnız asama-1

```
B1=$(printf 'asama-1' | openssl dgst -sha256 -mac HMAC -macopt hexkey:$A0 -binary | xxd -p -c 256)
echo "acc (ATLANMIS zincir) = $B1"
```

```
acc (ATLANMIS zincir) = bffc11620959254cef65b0e97fc8b54a1fe42cd6afe161b66aca5a758c1e17ff
```

`fc5c2a86...` ile `bffc1162...` **tamamen farklı** iki değerdir. Kritik işlem bu yanlış `acc` 'ten türettiği anahtarla gerçek sırrı **açamaz**; Demo 5'in SENARYO 2 ve 4'ünde gördüğümüz "ODEME REDDEDİLDİ ... decoy doner" sonucu tam olarak budur. Saldırganın `jmp` yaması **hangi kontrolü atladığını değiştirmiyor bile** — atlanan her kombinasyon farklı ama hep **yanlış** bir `acc` üretir, çünkü HMAC zinciri girdisindeki her değişikliği `çığ` etkisiyle yayar (Bölüm 3.1'deki aynı özellik, burada zincirlenmiş hâliyle).

✓ Kural

Kritik kararları **tek bir boolean'a** bağlamayın (yamalanır). Güvenlik kontrollerini bir **kontrol akışı sayacına** ve mümkünse bir **veri bağımlılığına** (kritik işlemin ürettiği sonucun anahtarına) bağlayın. Çift/örtüşen sayaçlar kullanın; başarısız çıkış noktalarını rastgeleleştirin (K15/K16).



Sık hatalar ve kontrol listesi

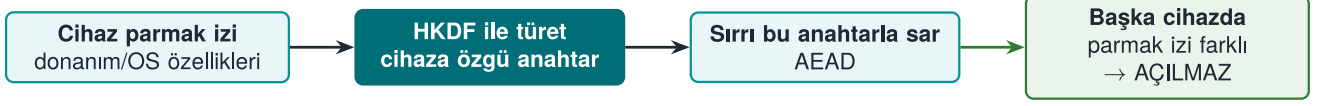
- ✓ Kritik karar tek bir `if` 'e mi bağlı, yoksa bir sayaca/anahtara mı?
- ✓ Kontrolleri **atlayan** bir yol gerçek sonucu üretebiliyor mu? (Üretmemeli.)
- ✓ Kontrol sırası önemli mi (sırayı bozan saldırı yakalanıyor mu)?
- ✓ Başarısızlıkta **decoy** mı dönüyor, yoksa hemen çöküp saldırıya ipucu mu veriyor?

10. Tepki (response) politikası, cihaz bağlama ve caydırma

Algılama tek başına işe yaramaz; asıl önemli olan **tepkidir**. Kötü bir tepki (ör. tespit anında `exit(1)`), saldırıya **tam olarak hangi kontrolün tetiklendiğini** söyler ve onu doğrudan o kontrolün yamasına yönlendirir. İyi bir tepki politikası saldırıyı **yavaşlatır ve yanıtlar**:

Cihaz bağlama: kopyalama işe yaramasın

çalınan dosya başka cihazda anlamsızdır



Code lifting ve kopyalama saldırılarının ölçeklenmesini bu engeller (11. hafta).

Tepki politikası: ne yapmalı?

tepki, algılamadan daha çok düşünülmesi gereken kısımdır

Hemen çök

saldırgana denetimin YERİNİ gösterir — kötü

Sessiz/gecikmeli tepki

sonraki işlemde sırrı sil, bağlantıyı kes

Decoy (sahte sonuç)

saldırgan yanlış yolda zaman kaybeder

Sunucuya bildir

telemetri + hesap/cihaz kısıtlama

Yanlış pozitifte meşru kullanıcıyı vurmaman, saldırıyı yormayan tepki seçilir.

Strateji	Ne yapar	Katalog
Fail-closed	Şüphede işlemi reddet, güven deposuna düşme	—
Sırrı sil	Tamper anında değerli veriyi güvenle sil (<code>kripto_temizle</code>)	K15
Decoy (sahte) çıktı	Çökmek yerine rastgele/sahte sonuç döndür; saldırıyı gerçekle sahteyi ayıramaz	K16
Gecikmeli/örtük tepki	Tepkiyi zaman/mesafe olarak tetikleyiciden ayır	K15
Cihaz/sürüm bağlama	Sırrı cihaza/sürüme bağla; anahtarlar kopyalansa bile başka cihazda açılmaz	L1/L2
Telemetri/attestation	Olayı sunucuya bildir; sunucu tarafı risk motoru karar versin	K5

Cihaz bağlama (Hafta 3, §13.1'in, Demo 9'un en iç kabuğu) burada RASP'in bir parçası olur: değerli sır, cihaz parmak izinden ve sürüm dizesinden **HKDF** ile türetilen bir anahtarla sarılır. Telefon klonlansa, dosyalar başka cihaza kopyalansa bile anahtar farklı çıkar ve sır **açılmaz**.

10.1 İşlenmiş örnek: cihaz parmak izinden anahtar türetmek (adım adım HKDF)

Demo 8'in "cihaz/sürüm bağlama" adımını **gerçek değerlerle** elle izleyelim. `kripto_hkdf_sha256`, **Hafta 3'te** gördüğümüz **RFC 5869 HKDF**'in iki adımını (Extract, Expand) uygular; burada `ikm` (girdi anahtar malzemesi) cihaz parmak izi, `tuz` (salt) sürüm dizesi, `info` ise sabit bir bağlam etiketidir:

Parametre	Değer (Demo 8'in normal senaryosu)
<code>ikm</code> (cihaz parmak izi)	<code>cihaz=SIM-MODEL-A;seri=SN-0001;uretici=DEMO</code>
<code>tuz</code> (sürüm)	<code>surum=1.0.0</code>
<code>info</code> (bağlam etiketi)	<code>rasp-veri-anahtari</code>

Adım 1 — Extract. Bu adım, uzunluğu ve rastgeleliği belirsiz olan `ikm`'i (cihaz parmak izi düz bir metindir, kriptografik olarak "iyi dağılmış" olmayabilir) sabit uzunlukta, güvenli bir ara anahtara sıkıştırır; bu ara anahtarın standart adı **PRK**'dir (Pseudo-Random Key — sözde rastgele anahtar): `PRK = HMAC-SHA256(anahtar = tuz, veri = ikm)`. `openssl` ile aynı hesabı yapalım:

HKDF Adım 1 - Extract (gerçek openssl çıktısı)

```
printf '%s' 'cihaz=SIM-MODEL-A;seri=SN-0001;uretici=DEMO' \
| openssl dgst -sha256 -mac HMAC -macopt hexkey:$(printf '%s' 'surum=1.0.0' | xxd -p -c 256) -
binary \
| xxd -p -c 256
```

```
PRK (gerçek cihaz) = a0aaf6995343abfb24d1d44d6aac18b20326fc5b9e84bdd82e89bc7ca99562ce
```

Adım 2 — Expand. İhtiyacımız 32 bayt (tek blok) olduğu için tek bir HMAC yeter: `T1 = HMAC-SHA256(anahtar = PRK, veri = info || 0x01)` (`0x01`, RFC 5869'daki blok sayacıdır). Bu `T1`, doğrudan 32 baytlık **veri anahtarı** olur:

```
anahtar (gerçek cihaz) = a19f06fd7b93894705615ac8933a2fa9e0b7cd290c7247a39e09dbc5be175d10
```

Adım 3 — aynı hesabı "başka cihaz" parmak iziyle tekrarlamak. Yalnız seri numarasını `SN-0001`'den `SN-9999`'a değiştirip (tıpkı bir saldırganın veri dosyasını başka bir telefona kopyalaması gibi) aynı iki adımı tekrarlayalım:

```
PRK (baska cihaz) = b6b88f3dce3ea3099c4ff0673d356c040689ebd72be4cc98fb158bfa4c0ac039
anahtar (baska cihaz) = b87e4dee6c5579a1456b977ce8cbf8d14f3363aa4e1c7c1a7198342c7689e220
```

Sonuç. İki anahtar (`a19f06fd...` ve `b87e4dee...`) **hiçbir ortak baytları olmayacak kadar** farklıdır — tek bir karakterlik seri numarası farkı, HKDF'in HMAC çekirdeği yüzünden (Bölüm 3.1'deki çığ etkisi) baştan sona farklı bir anahtar üretir. Demo 8'in SENARYO 3'ünde ("başka cihaz") gördüğümüz "cihaz/sürüm bağlama tutmadi" sonucu tam olarak budur: paketin şifrelendiği anahtar (`a19f06fd...`) ile çözmeye çalışılan anahtar (`b87e4dee...`) **aynı olmadığı için** AES-GCM'in kimlik doğrulama etiketi tutmaz ve `kripto_gcm_coz` başarısız döner — sır **hiçbir zaman açığa çıkmaz**, dosyalar başka cihaza kopyalansa bile.

Demo 8 — RASP motoru: algılama + tepki + cihaz/sürüm bağlama (capstone)

Demo 8 · `code/week-06/08-tamper-yanit` · haftanın capstone'u (K15/K16, L1/L2)

Önceki parçaları tek bir "öz koruma motoru"nda birleştirir: bir dizi kontrol çalışır; sır cihaz-bağlı bir anahtarla (HKDF + AES-GCM) sarılır; tamper algılanınca motor sırrı **siler**, tamper bayrağı kaldırır ve çökmek yerine **decoy** döndürür.

sh demo.sh — gerçek çıktı (kısaltılmış)

```
SENARYO 1 - Normal: kontroller gecer, cihaz dogru -> sir acilir
SONUC: TEMIZ. Sir acildi ve islem yapiliyor -> "ODEME-ANAHTARI-7C4A"
(sir kullanildikten sonra bellekten guvenle silindi)

SENARYO 2 - Tamper: bir kontrol basarisiz -> sil + decoy + bayrak
SONUC: TAMPER ALGILANDI -> algilama kontrolu basarisiz (tamper)
Politika: sir silindi, tamper bayragi kaldirildi, olay kaydedildi.
Cokmek yerine SAHTE (decoy) sonuc dondu: 60797327...

SENARYO 3 - Baska cihaz: kontroller gecer ama cihaz anahtari tutmaz
SONUC: TAMPER ALGILANDI -> cihaz/surum baglama tutmadi (klonlama?)
... decoy doner
```

Normal akışta kontroller geçti, cihaz doğruydı, sır açıldı, kullanıldı ve hemen silindi. Bir kontrol başarısız olunca (tamper) motor sırrı sildi ve decoy döndürdü. Paket **başka bir cihaza** taşındığında kontroller geçse bile cihaz-bağlı anahtar tutmadı ve sır açılmadı. Bu, Hafta 3'teki **güvenlik kabuğunun** en iç katmanının (cihaz bağlama) RASP ile birleşmiş hâlidir.

10.2 İşlenmiş örnek: aynı olaya üç farklı tepki, üç farklı sonuç

Bölüm 1.3'te aynı algılama sinyaline üç farklı tepkiyi kısaca karşılaştırmıştık; şimdi bunu Demo 8'in tamper senaryosu üzerinden **sonuç sonuç** karşılaştıralım. Olay: bir anti-debug kontrolü başarısız oldu (Demo 8 SENARYO 2).

Tepki	Saldırgana görünen	Sır ne olur?	Meşru kullanıcıya etkisi	Değerlendirme
(a) Hemen çök (<code>exit(1)</code> / <code>segfault</code>)	"Program çöktü" — hangi kontrolün tetiklendiğini, her kontrolü tek tek devre dışı bırakıp yeniden deneyerek birkaç turda bulur	Bellekte kalmışsa okunabilir	Nadir tetiklenirse fark etmez; sık tetiklenirse güvenilmez uygulama izlenimi verir	Saldırgana en çok bilgi veren, en kötü seçenek
(b) Gecikmeli sessiz (birkaç saniye/işlem sonra, sessizce oturumu sonlandır)	"Bir şey ters gitti ama ne zaman/nerede belli değil" — tetikleyici ile sonuç arasında zaman ve kod mesafesi var	Silinir (fail-closed)	Nadir görülen bir "oturum zaman aşımı" mesajı	Saldırganı yavaşlatır; makul bir orta seçenek
© Decoy (sahte sonuç) — Demo 8'in yaptığı	"İşlem başarılı görünüyor" — ama üretilen <code>decoy</code> sonucunun gerçek sırda hiçbir ilişkisi yok; saldırgan başarılı sandığı bir sonuçla vaktini boşa harcayabilir	Silinir, yerine rastgele veri döner	Hiç fark etmez (sır zaten meşru kullanıcı için doğru cihazda açılıyordu)	Saldırgana en az bilgi veren, en caydırıcı seçenek

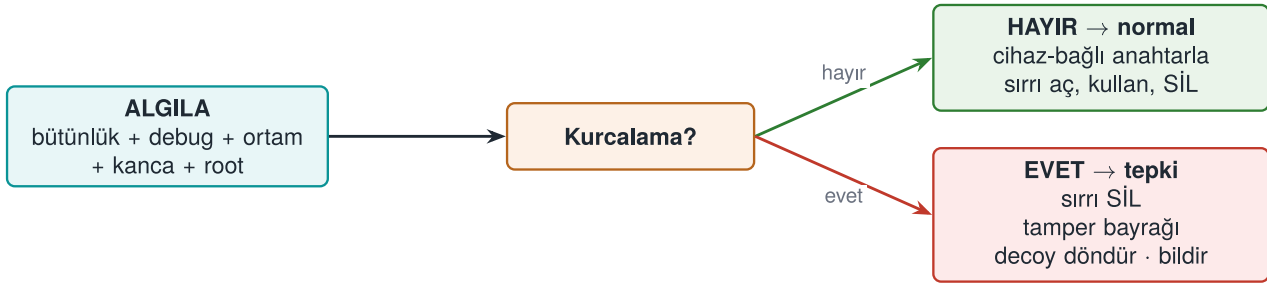
Üçü de **aynı algılama koduna** (Bölüm 1.3) dayanır; farkları yalnız **ne zaman ve ne döndürdükleridir**. Demo 8'in gerçek çıktısında © seçeneği uygulanır: "Cokmek yerine SAHTE (decoy) sonuc dondu: 60797327...". Bu sayı her çalıştırmada **farklıdır** (Alıştırma 7); sabit bir "HATA-0x1234" değeri olsaydı, saldırgan bu sabiti tanır ve gerçek sonuçtan ayırt ederdi – decoy'un rastgele olması bu ayrımı da engeller.

⚠ Sık hata: tepkiyi test ortamında hiç değiştirmemek

Geliştirme sırasında hızlı geri bildirim almak için genelde (a) seçeneği (hemen çök, konsola hata yaz) kullanılır – doğaldır, hata ayıklamak kolaylaşır. Sık yapılan hata, bu davranışı **üretime aynen taşımaktır**. Kural: tepki politikasını derleme **yapılandırmasına** bağlayın (hata ayıklama derlemesinde ayrıntılı hata mesajı, üretim derlemesinde sessiz/decoy); üretim ikili dosyasında hiçbir tespit mesajı **stdout/stderr'e** yazılmamalıdır – saldırgana ücretsiz bir hata ayıklama günlüğü vermeyin.

RASP motoru: her kritik işlemde

algıla → karar ver → davran



Tepki gecikmeli/sessiz olmalı: hemen çökmek saldırgana denetimin yerini gösterir.

✓ Kural

Tepkiyi tetikleyiciden **ayırın**: hemen `exit()` yapmak saldırgana yol gösterir. Tamper anında değerli veriyi **silin**, çökmek yerine **decoy** döndürün, olayı **sunucuya** bildirin ve sırrı **cihaza/sürüme bağlayın**. Algılama + tepki + bağlama üçünü birlikte kurun.

🔧 Sahada nasıl uygulanır? / Değerlendirici testi

Örnek mimaride ödeme anahtarı yalnız ödeme anında, cihaz-bağlı bir anahtardan açılır; tamper algılanınca anahtar imha edilir ve işlem sunucuya "şüpheli" olarak bildirilir. Değerlendirici tek bir kontrolü atlatıp **gerçek** sonuca ulaşmaya çalışır; korumaların **kombinasyon** hâlinde (tek tek değil, zincirleme) kırılması gerektiğini raporlar – bu, derinlemesine savunmanın çalıştığının kanıtıdır.

11. RASP'in sınırları ve etik

RASP'in sınırları: ne vaat eder, ne etmez?

cihazın sahibi saldırgansa yeterli zamanla her katman aşılır

ETMEZ

kırılmazlık
MATE'ye karşı kalıcı engel
kötü mimariyi kurtarma
meşru kullanıcıyı ayırt etme garantisi

EDER

saldırıyı geciktirir
toplu kırılmayı zorlaştırır
telemetriyle görünürlük
riskli ortamda kapsamı daraltır

Etik sınır: kullanıcının cihazına zarar vermeyin, veri toplamayı şeffaf ve ölçülü tutun.

✗ 'RASP koydum, uygulamam kırılmaz.'

Cihazın sahibi saldırgansa (MATE), yeterli zamanla **her** istemci tarafı koruma atlatılabilir. RASP kırılmazlık sağlamaz; saldırıyı **pahalı, ölçeklenemez ve gürültülü** kılar. Nihai güvence her zaman **sunucu tarafındadır** (risk motoru, anomali tespiti, anahtar yenileme, işlem limitleri).

✗ 'Tek bir güçlü kontrol yeter.'

Her tek kontrolün bir karşı-teknik vardır (yama, kanca, sahteleme). Güç **kombinasyondan** gelir: çok sayıda bağımsız sinyal + kontrol akışı sayacı + cihaz bağlama + sunucu attestation.

✗ 'Tespit edince hemen çökmeliyim.'

Hemen çökmek saldırgana hangi kontrolün tetiklediğini söyler. Tepkiyi **geciktirin ve örtün**: decoy döndürün, olayı sessizce bildirin, sırrı silin.

✗ 'Ortam VM ise kesinlikle saldırıdır.'

WSL2/Hyper-V/VBS yüzünden çoğu meşru makine hipervizör bildirir (Demo 3). Ortam sinyalleri **risk skoruna** katkıdır, ikili karar değil; yoksa gerçek kullanıcıları engellersiniz.

✗ 'RASP her yavaşlığa değer.'

RASP bir **maliyet** getirir (CPU, batarya, karmaşıklık, yanlış pozitif). Korumayı **varlığın değerine** göre ölçekleyin: her fonksiyona değil, kritik işlemlere (ödeme, anahtar kullanımı) koyun. Bu, [Hafta 1](#)'deki "korumayı varlığa göre planla" ilkesidir.

✗ 'Risk skoru eşiğini bir kere ayarlayıp unuturum.'

Cihazlar, işletim sistemi sürümleri ve saldırı araçları zamanla değişir; bugün "kesin saldırgan" sayılan bir skor, altı ay sonra yaygınlaşan meşru bir yapılandırmayı (yeni bir sanallaştırma özelliği, yaygınlaşan bir geliştirici aracı) yanlışlıkla yakalayabilir. Eşığı ve ağırlıkları **telemetriyle** (Bölüm 10.2'deki sunucu bildirimi) düzenli gözden geçirin (Bölüm 5.1).

12. Dönem projesi: bu hafta

Bu hafta güvenlik kılavuzunuzun **RASP (öz koruma)** bölümlerini yazacak ve ilk RASP kontrolünüzü projenize ekleyeceksiniz. Bu, **vize proje gösteriminin** (Hafta 7) ve **Quiz-1'in** (Hafta 8, 1–6. haftalar) doğrudan konusudur; ara proje rubriğinde **A2.4 RASP Teknikleri (15p → ÖÇ.3)**, final rubriğinde **F2.4 İkili Uygulama Korumaları (15p → ÖÇ.3)** olarak puanlanır.

- ✓ **S10 — RASP algılama envanteri:** Projenizdeki her kritik işlem için hangi RASP kontrollerini (bütünlük, anti-debug, ortam, kanca, root, imza) uyguladığınızı, her birinin **neyi** algıladığını ve **nasıl atlatılabildiğini** tabloya dökün.
- ✓ **S10 — Tepki politikası:** Tamper algılandığında ne olacağını yazın: hangi sır silinir, fail-closed mı, decoy mu, cihaz/sürüm bağlama var mı, sunucuya bildirim var mı.

S10 RASP envanteri sablonu

Kontrol	Neyi algılar	Nasıl atlatılır	Güçlendirme
Butunluk (HMAC)	İkili yama	Checker yamalanır	Ortusen denetleyici
Anti-debug	Hata ayıklayıcı	ptrace no-op / yama	Coklu sinyal+sayac
Kanca algılama	LD_PRELOAD/Frida	maps gizleme	dladdr + maps
Cihaz bağlama	Klonlama	(cihaza bağlı)	HKDF + AES-GCM

S10 tepki politikası sablonu

Tetikleyici	Tepki	Bildirim
butunluk basarisiz	sirri sil + decoy + fail-close	sunucuya olay
cihaz tutmaz	ac-ma, decoy	sunucuya olay

🔥 Kurs gereksinim aileleri

Bu bölümler şu gereksinim ailelerini karşılar: CEN429-RA (RASP algılama), CEN429-RT (tepki politikası), CEN429-BD (cihaz/sürüm bağlama), CEN429-CF (kontrol akışı bütünlüğü). Her gereksinim için S17 uyum matrisine bir satır ekleyin: gereksinim → durum → belge bölümü → kanıt (test/kod/demo).

13. Sınıf etkinlikleri

Etkinlik 1 — Kancayı kendi elinizle kurun (15 dk, ikili, WSL)

Demo 4'ü WSL'de çalıştırın. Sonra `sahtekanca.c`'ye `getenv`'i de ekleyin (`LD_PRELOAD`'ı gizlemek için boş döndürsün). Yeniden derleyip çalıştırın: 1. sinyal (`getenv`) artık yakalıyor mu? 2. sinyal (`dldaddr`) hâlâ yakalıyor mu? Bu neden daha sağlam?

Etkinlik 2 — Tek `if`'i yamalayın (10 dk, tartışma)

Demo 5'te `atlat` modu gerçek sonucu üretemedi. Şimdi düşünün: kritik işlem yalnız `if (kontroller_gecti) onayla;` olsaydı, saldırgan tek bir `jmp` ile ne yapardı? Anahtar zinciri neden bu yamayı işe yaramaz kılar?

Etkinlik 3 — Yanlış pozitif avı (15 dk, 3-4 kişilik grup)

Demo 3'ü çalıştırın; muhtemelen "hipervizör VAR" diyecek. Bir uygulama sırf bunun için çalışmayı reddetseydi hangi meşru kullanıcılar mağdur olurdu? Root algılama (Demo 7) için de aynı soruyu tartışın. RASP kararları neden **risk skoru** olmalı, ikili "çalış/çalışma" değil?

Etkinlik 4 — Tepki politikası tasarımı (10 dk, bireysel)

Bir mobil bankacılık uygulaması root algıladı. Üç farklı tepki tasarlayın: (a) hemen çök, (b) işlemi reddet ve kullanıcıyı bilgilendir, © sessizce çalışmaya devam et ama sunucuya bildir ve işlem limitini düşür. Hangisi saldırgana **en az** bilgi verir? Hangisi meşru kullanıcıya en az zarar verir?

14. Kod okuma alıştırmaları

Okuma 1 — Bu anti-debug güvenli mi?

```
if (IsDebuggerPresent()) {
    exit(1);
}
do_payment();
```

Cevap

Hayır. (1) Tek bir sinyale bağlı; (2) sonuç doğrudan `exit()` — saldırgan `IsDebuggerPresent`'in dönüşünü ya da bu `if`'i yamalar; (3) hemen çıkmak hangi kontrolün tetiklediğini söyler. Düzeltme: birden çok sinyal + kontrol akışı sayacı + gecikmeli/örtük tepki (Demo 2, 5, 8).

? Okuma 2 — Bu bütünlük denetimi neden zayıf?

```
uint32_t crc = crc32(kod, kod_boy);
if (crc != BEKLENEN_CRC) tamper();
```

✓ Cevap

CRC32 kriptografik değildir: saldırgan kodu yamalıyıp CRC'yi eşleyecek şekilde kolayca ayarlar. Ayrıca tek bir denetleyici var (kendisi yamalanabilir) ve sonuç bir davranışa bağlı değil. Düzeltme: HMAC/imza + örtüşen denetleyiciler + veri bağımlılığı (Demo 1).

? Okuma 3 — Bu kanca algılama neye kaniyor?

```
if (getenv("LD_PRELOAD") == NULL) printf("temiz\n");
```

✓ Cevap

getenv 'in kendisi kancalanıp NULL döndürebilir; ya da saldırgan LD_PRELOAD yerine ptrace /PLT kancası kullanır. Düzeltme: fonksiyonun **kaynağını** doğrula (dlsym + dladdr), /proc/self/maps tara, birden çok sinyal kullan (Demo 4).

? Okuma 4 — Bu ortam algılama kararı neden riskli?

```
unsigned int ecx;
__cpuid_ecx(1, &ecx);
if (ecx & (1u << 31)) {
    /* hipervizör biti set */
    exit(1);          /* "analiz ortamı, çalışmayı reddet" */
}
```

✓ Cevap

Hipervizör biti tek başına **kanıt değildir** (Bölüm 5, Demo 3): WSL2, Hyper-V ve sanallaştırma tabanlı güvenlik (VBS) yüzünden birçok **meşru** Windows makinesi de bu biti set eder. Bu kod, sırf bir hipervizör gördüğü için milyonlarca gerçek kullanıcıyı engelleyebilir — klasik bir yanlış pozitif. Düzeltme: bu sinyali tek başına karar vermek yerine Bölüm 5.1'deki gibi bir **risk skoruna** ekleyin ve kararı diğer sinyallerle (satıcı imzası, zamanlama, hata ayıklayıcı) birlikte, bir **eşiğe** göre verin.

15. Kendi başına çalış

Bu alıştırmalar not verilmez; pekiştirme içindir. Hepsini kendi bilgisayarınızda, `code/week-06` demoları üzerinde yapılı.

? Alıştırma 1 — Kolay: Farklı baytı yamalayın

Demo 1'de kopyanın **farklı** bir ofset baytını değiştirin. HMAC yine değişiyor mu? Neden (çığ etkisi)?

? Alıştırma 2 — Kolay: Altın değeri kurcalayın

Demo 1'de `cikti/altin.hmac` dosyasını saldırgan gibi düzenleyin. Denetim artık neyi yakalayamaz? Bu, "altın değeri nerede saklanmalı?" sorusunu neden önemli kılar?

? Alıştırma 3 — Orta: gdb altında Demo 2

Demo 2'yi `strace -f ./bin/linux/antidebug` ile çalıştırın. TracerPid ne oluyor? Ana sürecin adı ne? Hangi sinyaller tetikleniyor?

? Alıştırma 4 — Orta: getenv'i de kancalayın

Demo 4'ün `sahtekanca.c`'sine `getenv` ekleyin. Hangi sinyal kör oluyor, hangisi hâlâ yakalıyor? (Etkinlik 1'in kod hâli.)

? Alıştırma 5 — Orta: Zincire beşinci kontrol

Demo 5'te `ASAMA` 'yı 5 yapın ve iki kontrol noktası daha ekleyin. Zincir nasıl değişir? Normal akış hâlâ onaylanıyor mu?

? Alıştırma 6 — Orta: Karşılıklı doğrulama

Demo 6'yı genişletin: modül dosyası, native'in (kendi ikilinizin) HMAC'ini de taşıyın; native modülü, modül de native'i doğrulasın (K6). Saldırganın işi neden zorlaştı?

? Alıştırma 7 — Zor: Decoy'u ölçün

Demo 8'de tamper senaryosunda dönen decoy'u iki kez çalıştırıp karşılaştırın. Aynı mı, farklı mı? Neden decoy'un **rastgele** (her seferinde farklı) olması, sabit bir "hata değeri"nden daha iyidir?

? Alıştırma 8 — Zor: Cihaz bağlamayı test edin

Demo 8'de `baska-cihaz` modunda sırrı açmanın bir yolu var mı? Cihaz parmak izini değiştirmeden anahtarı türetmek mümkün mü? Bu, çalınan bir veri dosyasını başka telefonda neden işe yaramaz kılar?

? Alıştırma 9 — Zor: Timing eşiği

Demo 3'ün zamanlama ölçümünü Demo 2'deki gibi `gdb` altında (tek adımlı) çalıştırın. Süre nasıl değişiyor? Güvenilir bir eşik koymak neden zordur (makineye/yüke bağımlılık)?

? Alıştırma 10 — Orta: RASP'i varlığa göre ölçekleyin

Projenizde hangi işlemler RASP'i hak ediyor (ödeme, anahtar kullanımı) ve hangileri gereksiz (yerel ayar okuma)? Neden her fonksiyona RASP koymak kötü bir fikirdir?

? Alıştırma 11 — Orta: Kendi risk skorunuzu kurun (Bölüm 5.1)

Demo 3, Demo 2 ve Demo 7'nin ürettiği sinyalleri (hipervizör biti, TracerPid, root göstergesi) bir araya getirip Bölüm 5.1'deki gibi bir ağırlıklı toplam yazın. Kendi bilgisayarınızda (temiz) skor kaç çıkıyor? `gdb` altında çalıştırırsanız kaç çıkıyor? Bankacılık uygulaması için önerdiğiniz eşik kaç olurdu, neden?

? Alıştırma 12 — Zor: HKDF'i elle tekrar hesaplayın (Bölüm 10.1)

Bölüm 10.1'deki `openssl` komutlarını kendi makinenizde çalıştırın; aynı `PRK` ve anahtar değerlerini elde ediyor musunuz? Şimdi `SURUM` dizesini (`surum=1.0.0` yerine `surum=1.0.1`) değiştirip anahtarı tekrar hesaplayın. Bu, bir sürüm yükseltmesinin eski verideki sırları neden "kilitlediğini" nasıl açıklar?

16. Kendini sinama

? 1. RASP nedir? WAF'tan farkı?

Uygulamanın içine gömülü, çalışma zamanında kendini izleyen koruma. WAF uygulamanın **önünde** gelen istekleri süzer; RASP uygulamanın **içinde** kendi bütünlüğünü, belleğini ve ortamını izler.

? 2. Algılama → savunma → caydırma üçlüsünü açıklayın.

Algılama: bir şeyin yanlış olduğunu anlamak (yama, debugger, kanca, root). Savunma: anlamlı yanıt (fail-closed, sil, cihaza bağla). Caydırma: saldırıyı pahalılaştırmak (decoy, gecikme, telemetri).

? 3. MATE saldırgan modeli nedir?

"Man-At-The-End": saldırgan uygulamanın çalıştığı uç noktanın sahibidir; belleği okuyabilir, kodu değiştirebilir, hata ayıklayıcı bağlayabilir. Beyaz kutu (Hafta 11) ile aynı varsayım.

? 4. Self-hashing nedir? Neden CRC değil HMAC?

Uygulamanın kendi kod/dosyasının özetini çalışma zamanında hesaplayıp altın değerle karşılaştırması. CRC kriptografik değildir (saldırgan eşler); HMAC için gizli anahtar gerekir.

? 5. Bütünlük denetimi tek başına neden yetmez?

Denetleyicinin kendisi de yamalanabilir. Çözüm: örtüşen denetleyiciler, sonucu veri bağımlılığı yapmak, sonucu attestation.

? 6. Linux'ta ve Windows'ta bir hata ayıklayıcı nasıl algılanır?

Linux: `/proc/self/status` TracerPid (>0). Windows: `IsDebuggerPresent` (PEB BeingDebugged), `CheckRemoteDebuggerPresent`, `PEB NtGlobalFlag`.

? 7. Hipervizör biti neden zayıf bir sinyaldir?

WSL2/Hyper-V/VBS yüzünden çoğu meşru makine hipervizör bildirir; tek başına "analiz ortamı" demek değildir. Yanlış pozitif riski yüksek.

? 8. LD_PRELOAD kancası nasıl algılanır?

Kritik fonksiyonun **kaynağını** doğrulayarak: `dlsym(RTLD_DEFAULT) + dlsym` ile fonksiyonu sağlayan `.so` 'ya bak; `libc/vDSO/ld` dışı bir modülse kanca vardır. `getenv(LD_PRELOAD)` tek başına zayıftır.

? 9. Root/jailbreak göstergeleri neden 'kanıt değil, sinyal'dir?

Kolayca sahtelenir (Magisk Hide); ayrıca meşru root kullanıcıları engelleyebilir. Modern yol: platform attestation (sunucu tarafı, nonce'lu).

? 10. APK imza doğrulaması neyi engeller? C'deki karşılığı?

Yeniden paketlemeyi (repackaging): değiştirilmiş uygulama orijinal imzayı taşıyamaz. C'de karşılığı: bir bileşeni yüklemekten önce HMAC/imza ile doğrulamak (Demo 6).

? 11. Kontrol akışı sayacı neyi çözer?

Tek bir `if` kontrolünün yamalanıp atlanmasını. Kritik işlem doğru sonucu ancak bütün kontrollerden sırayla geçmişse (doğru anahtar zinciri) üretir.

? 12. İyi bir tamper tepkisi neden hemen `exit()` değildir?

Hemen çökmek saldırgana hangi kontrolün tetiklediğini söyler. İyi tepki: sil + decoy + gecikmeli/örtük + sunucuya bildirim.

? 13. Decoy (sahte) çıktı neden çökmekten iyidir?

Saldırgan gerçek ile sahteyi kolayca ayırt edemez; hangi kontrolün tetiklediğini gizler; veri sızmaz.

? 14. Cihaz bağlama RASP'e ne katar?

Anahtarlar/dosyalar kopyalansa bile başka cihazda açılmaz (klonlamaya karşı). HKDF(cihaz parmak izi)+AES-GCM ile kurulur; [Hafta 3](#) güvenlik kabuğunun en iç katmanı.

? 15. RASP kırılmaz mıdır? Öyleyse neden kullanılır?

Cihazın sahibi saldırgansa nihai olarak kırılabilir. Amaç kırılmazlık değil; saldırıyı pahalı, ölçeklenemez ve gürültülü kılmak + sunucu tarafı denetimle birleştirmek.

? 16. RASP'i her fonksiyona koymak neden kötü fikir?

Maliyet (CPU, batarya, yanlış pozitif, karmaşıklık) getirir. Korumayı varlığın değerine göre ölçekleyin: kritik işlemlere (ödeme, anahtar) koyun.

? 17. Ortam algılamada risk skoru yaklaşımı nedir, neden ikili karardan daha iyidir?

Her sinyale bir ağırlık verilir, ağırlıklar toplanır ve bir eşikle karşılaştırılır (Bölüm 5.1). İkili karar (tek sinyalle çalış/çalışma) tek bir zayıf sinyalle (ör. hipervizör biti) meşru kullanıcıları engeller; risk skoru birden çok bağımsız sinyalin birlikte görünmesini ister.

? 18. TracerPid alanı nereden okunur ve neyi temsil eder?

Linux'ta `/proc/self/status` dosyasındaki bir satırdır; süreci `ptrace()` ile izleyen sürecin PID'ini tutar. `0` ise izleyen yok; `>0` ise o PID süreci izliyor demektir (Bölüm 4.1).

? 19. `dlsym` ve `dlsym` RASP'te birlikte nasıl kullanılır?

`dlsym(RTLD_DEFAULT, ad)` bir sembolün çalışan süreçteki **adresini** bulur; `dlsym` o adresi hangi `.so` dosyasının sağladığını söyler. Beklenen kütüphane (`libc/vDSO/ld`) yerine yabancı bir `.so` çıkarsa kanca vardır (Bölüm 6.1).

? 20. HKDF'in Extract ve Expand adımları cihaz bağlamada nasıl kullanılır? ✓

Extract: `PRK = HMAC(anahtar=sürüm/tuz, veri=cihaz parmak izi)`. Expand: `anahtar = HMAC(PRK, info || sayaç)`. Farklı bir cihaz parmak izi tamamen farklı bir PRK ve dolayısıyla farklı bir anahtar üretir; sır başka cihazda açılmaz (Bölüm 10.1).

? 21. Aynı tamper olayına üç farklı tepki (hemen çök, gecikmeli sessiz, decoy) arasındaki temel fark nedir? ✓

Saldırgana verdikleri bilgi miktarı ve zamanlaması: hemen çökmek en çok bilgi verir (hangi kontrolün tetiklendiğini gösterir), gecikmeli sessiz tepki bunu zaman/mesafe olarak bulanıklaştırır, decoy ise saldırgana "başarılı" görünen ama işe yaramaz bir sonuç vererek en az bilgi verir (Bölüm 10.2).

? 22. `KorunanSayac` yapısındaki gölge kopya ve özet alanları neden ayrı tutulur? ✓

Saldırgan yalnız `deger` alanını bellekte değiştirirse, `golge` (maskelenmiş kopya) ve `ozet` (kısa MAC) eski değerlerle tutarsız kalır; okuma fonksiyonu bu tutarsızlığı yakalar (Bölüm 7.1).

? 23. Kontrol akışı sayacındaki `acc` zincirinin çığ etkisiyle ilişkisi nedir? ✓

Her aşamada `acc`, bir önceki `acc`'yi anahtar olarak kullanan bir HMAC ile güncellenir. Bir aşama atlanır ya da sırası değişirse, HMAC'in çığ etkisi yüzünden sonuç **tamamen farklı** bir `acc` / anahtar üretir; tek bir `jmp` yaması hangi kombinasyonu denerse denesin gerçeğe ulaşamaz (Bölüm 9.1).

? 24. Bir ürünün risk iştahı (risk appetite) tepki eşiğini nasıl belirler? ✓

Yanlış pozitifin maliyeti yüksekse (bankacılık — işlem tamamlanamaz, para kaybı riski) eşik düşük tutulur: şüphede reddet. Yanlış negatifin maliyeti düşükse (sadakat uygulaması) eşik yüksek tutulur: nadiren reddet (Bölüm 5.1).

? 25. Root göstergesini tek bir `if` ile "reddet" yapmanın sakıncası nedir? ✓

Gerçek saldırganı yakalamaz (ipuçlarını zaten gizlemiştir) ama meşru biçimde root'lamış sıradan kullanıcıyı cezalandırır. Kural: göstergeyi bir risk skoruna ekleyin, kararı eşiğe bırakın (Bölüm 8.1.1).

? 26. Üretim ikili dosyasında tespit mesajlarının `stdout` / `stderr` 'e yazılmaması neden önemlidir? ✓

Yazılırsa saldırgana ücretsiz bir hata ayıklama günlüğü/ipucu verilmiş olur — hangi kontrolün ne zaman tetiklendiğini doğrudan gösterir. Tepki politikası derleme yapılandırmasına bağlanmalı, üretimde sessiz kalmalıdır (Bölüm 10.2).

17. Quiz-1 tarzı örnek sorular

Quiz-1 (8. hafta) 1–6. haftaları kapsar; bu tarz kısa soru + kod okuma içerir

1. (Çoktan seçmeli) Aşağıdakilerden hangisi bir **yamayı (patch)** çalışma zamanında algılar? a) ECB · b) **HMAC ile self-hashing** · c) PBKDF2 · d) TOFU
2. (Doğru/Yanlış) "Hipervizör biti set ise cihaz kesinlikle bir analiz ortamıdır." → **Yanlış** (WSL2/VBS meşru makinelerde de set eder).
3. (Kısa cevap) `if (IsDebuggerPresent()) exit(1);` neden zayıftır? → Tek sinyal + doğrudan `exit()`; yamalanır ve tetikleyiciyi ele verir.
4. (Kod okuma) Bir kritik işlem yalnız `if (checks_ok)` ile korunuyorsa saldırgan onu nasıl atlar ve **kontrol akışı sayacı** bunu neden engeller? → Tek `jmp` yaması; sayaç/anahtar zinciri sonucu veri bağımlılığı yapar.
5. (Eşleştirme) TracerPid ↔ anti-debug · dladdr ↔ kanca algılama · HKDF(cihaz) ↔ cihaz bağlama · decoy ↔ tamper tepkisi · APK imzası ↔ repackaging.
6. (Kısa cevap) Bir ortam algılama denetimi "hipervizör VAR" derse, uygulama işlemi doğrudan reddetmelidir/reddetmemelidir? Neden? → Reddetmemeli; bu tek başına zayıf bir sinyaldir (WSL2/VBS), bir **risk skoruna** katkı olarak kullanılmalıdır (Bölüm 5.1).
7. (Doğru/Yanlış) "HMAC ile bütünlük denetimi tek başına, checker'ın kendisinin yamalanmasına karşı korur." → **Yanlış**; bu yüzden örtüşen denetleyiciler ve sonucu veri bağımlılığı yapmak gerekir (Bölüm 3).
8. (Kısa cevap) Cihaz bağlamada `SURUM` (sürüm) dizesi neden HKDF'e `tuz` (salt) olarak veriliyor? → Sürüm değişince türetilen anahtar da değişsin diye; bu, eski sürümdeki bir zafiyetle üretilmiş veriyi yeni sürümde otomatik olarak "kilitler" (Bölüm 10.1, Alıştırma 12).

18. Kaynaklar ve ileri okuma

Ders kitabı — J. Viega, M. Messier, *Secure Programming Cookbook for C and C++*, O'Reilly, 2003:

- Tarif 12.2 (Detecting Modification — CRC → HMAC/imza olarak güncellendi)
- Tarif 12.12–12.15 (Detecting Debuggers — Unix/Windows; SoftICE → modern araçlar)
- Bölüm 12 (Anti-Tampering) genel olarak bu haftanın ve [Hafta 9](#)'un çekirdeğidir.

Açık standartlar ve kaynaklar

- OWASP MASVS-RESILIENCE ve MASTG (mobil dayanıklılık: anti-debug, anti-tamper, root/emülatör algılama, imza doğrulama).
- Android APK Signature Scheme v2/v3 (yeniden paketlemeye karşı imza); Play Integrity / donanım destekli attestation.
- MITRE CWE: CWE-693 (Protection Mechanism Failure), CWE-919 (mobil zayıflıklar), CWE-925 (uygunsuz doğrulama).
- SEI CERT C: ENV ve FIO kuralları (ortam ve dosya güvenli kullanımı).

**Sözlük**

Terim	Türkçe	Kısa tanım
RASP	Çalışma zamanı öz koruması	Uygulamanın içine gömülü, çalışırken kendini koruyan mekanizmalar
Detection / Defense / Deterrence	Algılama / Savunma / Caydırma	RASP'in üç işlevi
MATE	Uçtaki saldırgan	Cihazın sahibi olan saldırgan (beyaz kutu)
Self-hashing	Kendi kodunun özetini denetleme	Uygulamanın kendi kod/dosyasını çalışma zamanında özetleyip doğrulaması
Tamper	Kurcalama	İkili/veri üzerinde yetkisiz değişiklik
Patch (yama)	Yama	İkili kodu değiştirerek davranışı değiştirme
Anti-debug	Hata ayıklayıcı karşıtı	Bağlı hata ayıklayıcıyı algılama/engelleme
TracerPid	İzleyen süreç kimliği	Linux'ta bir süreci <code>ptrace</code> ile izleyen sürecin PID'i
Hooking (kanca)	Kanca takma	Bir fonksiyonu kendi sürümüyle değiştirme (LD_PRELOAD, PLT/GOT, inline)
DBI / Frida	Dinamik ikili enstrümantasyon	Çalışan programa kod enjekte edip davranışını değiştirme
Emulator / VM detection	Emülatör/VM algılama	Analiz ortamını ipuçlarından sezme (CPUID, timing)
Root / jailbreak	Kök erişimi	Cihazın güvenlik kısıtlarının kaldırılmış olması
Attestation	Platform doğrulaması	Cihaz/uygulama bütünlüğünün (çoğu zaman sunucu tarafı) kanıtı
Repackaging	Yeniden paketleme	Uygulamayı açıp değiştirip yeniden paketleme
Control-flow integrity	Kontrol akışı bütünlüğü	Kritik akışın atlanamaz/sıralı olmasını sağlama (sayaç/anahtar zinciri)
Tamper response	Kurcalama tepkisi	Tamper anındaki politika (sil, fail-closed, decoy)
Decoy	Sahte çıktı	Çökmek yerine döndürülen, gerçekten ayırt edilemez sahte sonuç
Device / version binding	Cihaz / sürüm bağlama	Anahtarı belirli cihaza/sürüme bağlayıp başka yerde açılmaz kılma
Fail-closed	Hata durumunda reddetme	Şüphede işlemi reddetme (güven deposuna düşmeme)

Terim	Türkçe	Kısa tanım
Risk skoru / eşik	Risk puanı / sınır değer	Birden çok zayıf sinyalin ağırlıklı toplamı; karar bu toplamın bir eşiği geçip geçmediğine bakar
Risk iştahı (risk appetite)	Risk toleransı	Ürünün yanlış pozitif ile yanlış negatif arasında kabul ettiği denge
<code>dlsym</code> / <code>dladdr</code>	Sembol çözme / adres bilgisi	Bir fonksiyon adının bellek adresini bulma / bir adresin hangi <code>.so</code> 'dan geldiğini bulma
vDSO	Sanal dinamik paylaşımlı nesne	Çekirdeğin süreç belleğine kendiliğinden eklediği, sistem çağırısı yapmadan hızlı okuma sağlayan sanal <code>.so</code>
PRK	Sözde rastgele anahtar	HKDF'in Extract adımından çıkan, Expand adımına girdi olan ara anahtar
HKDF Extract / Expand	Sıkıştırma / genişletme	HKDF'in iki adımı: düzensiz girdiyi PRK'ya sıkıştırma, PRK'dan istenen uzunlukta anahtar türetme
Çığ etkisi (avalanche effect)	Çığ etkisi	Girdide tek bir bit değişince çıktının ortalama yarısının değişmesi; kriptografik özet/MAC'in temel özelliği
<code>PR_SET_DUMPABLE</code>	Dökülebilirlik bayrağı	Linux'ta bir sürecin başka bir süreç tarafından <code>ptrace</code> ile izlenip izlenemeyeceğini ve çekirdek dökümü üretip üretmeyeceğini kontrol eden bayrak

Bir sonraki hafta

7. hafta — Ara proje gösterimleri. Bu haftaki RASP kontrollerinizi (bütünlük denetimi, hata ayıklayıcı/ortam/kanca algılama, kontrol akışı sayacı, tepki politikası) projenizde canlı göstereceksiniz. **8. hafta**, 1–6. haftaları kapsayan **Quiz-1**'dir. Ardından **9. hafta — Gelişmiş gizleme ve çeşitlendirme**, bu haftanın hata ayıklayıcı/ortam algılama ve "gizleme/caydırma kırılmazlık değildir" fikirlerini kod gizleme tarafında derinleştirerek sürer.